

Introduction to Algorithm

기초 알고리즘

Binary Search

Binary Search

순차 탐색 (Linear Search)

23	34	36	41	45	49	52	57	64	72	76	81	89
----	----	----	----	----	----	----	----	----	----	----	----	----

반복문을 사용해서 배열에서 값을 찾을 때,
처음부터 시작해서 순차적으로 값을 찾음

이런 방식으로 값을 찾는 것을 순차 탐색이라고 함



순차 탐색 (Linear Search)

그러나 순차 탐색은, 배열 내에 있는 **모든 값을 체크**해야 하기 때문에,
데이터의 양이 많아지면 너무 오랜 시간이 소요될 수 있음

배열 전체를 탐색하지 않아도 원하는 값을 찾아 낼 수 있는

이진 탐색 알고리즘에 대해 알아보자



이진 탐색 (Binary Search)

23	34	36	41	45	49	52	57	64	72	76	81	89
----	----	----	----	----	----	----	----	----	----	----	----	----

45를 찾는다고 할 때,

어떤 방식으로 빠르게 탐색할 수 있을지 알아보자



이진 탐색 (Binary Search)

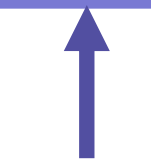
target: 45

left: 0

right: 12

mid:

23	34	36	41	45	49	52	57	64	72	76	81	89
----	----	----	----	----	----	----	----	----	----	----	----	----



left

right



먼저, 양쪽 끝 지점을 **left**와 **right**라고 부르자



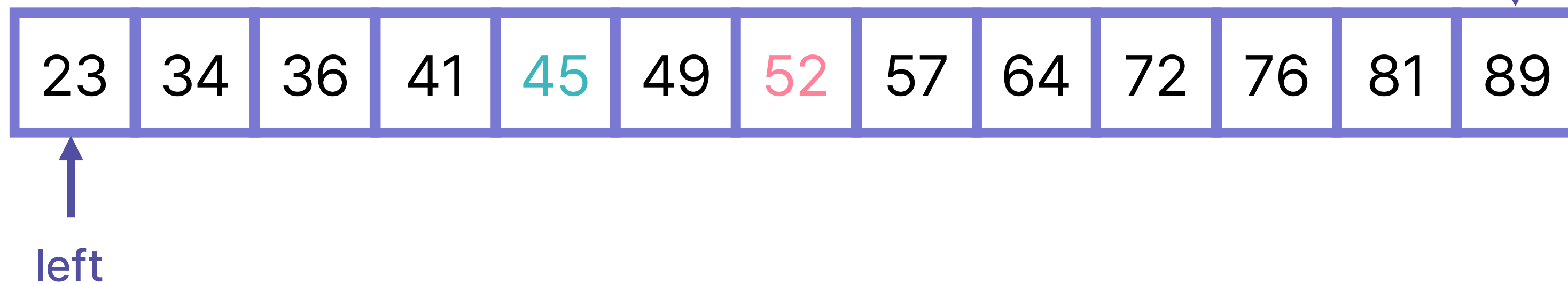
이진 탐색 (Binary Search)

target: 45

left: 0

right: 12

mid: $(0 + 12) / 2 = 6$



left와 right가 정해지면,

두 지점의 가운데 지점을 mid라고 부르자

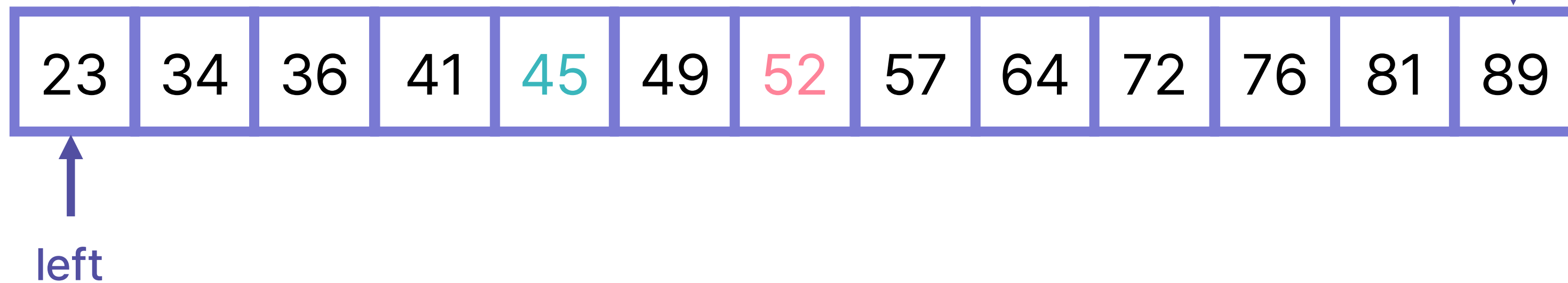
이진 탐색 (Binary Search)

target: 45

left: 0

right: 12

mid: 6



이제, **mid** 위치에 있는 값이

찾는 값보다 큰지, 작은지, 아니면 같은지 확인해야 함



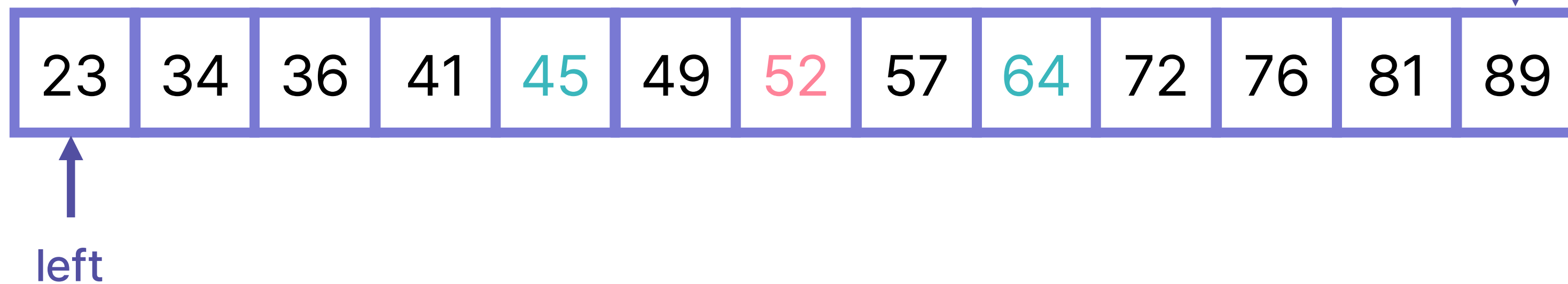
이진 탐색 (Binary Search)

target: 45

left: 0

right: 12

mid: 6



만약 **mid** 값이 찾는 값보다 작으면 **left**를 움직이고,
찾는 값보다 같거나 크다면 **right**를 움직임

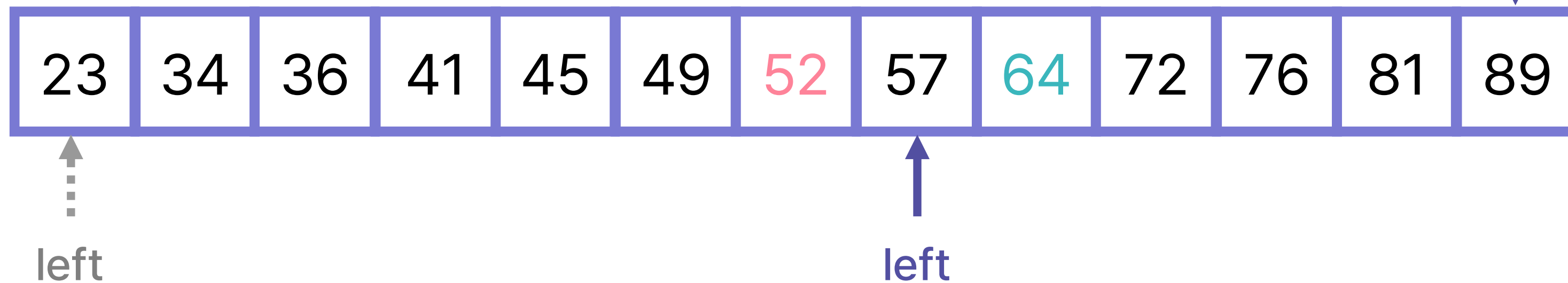
이진 탐색 (Binary Search)

target: 64

left: $\text{mid} + 1 = 7$

right: 12

mid: 6



left를 움직여야 한다면 mid 값이 찾는 값보다 작다는 것이니,

left를 mid보다 한 칸 앞으로 보내줌

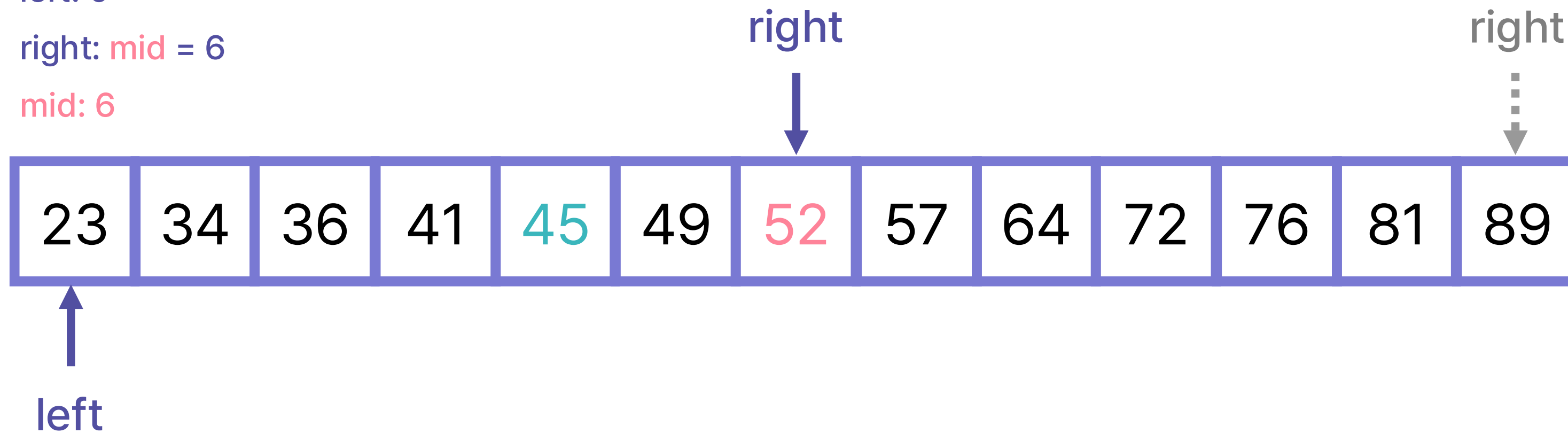
이진 탐색 (Binary Search)

target: 45

left: 0

right: mid = 6

mid: 6



반대로 **right**를 움직여야 한다면, 같거나 큰 것이므로

right를 **mid** 자리로 옮겨줌

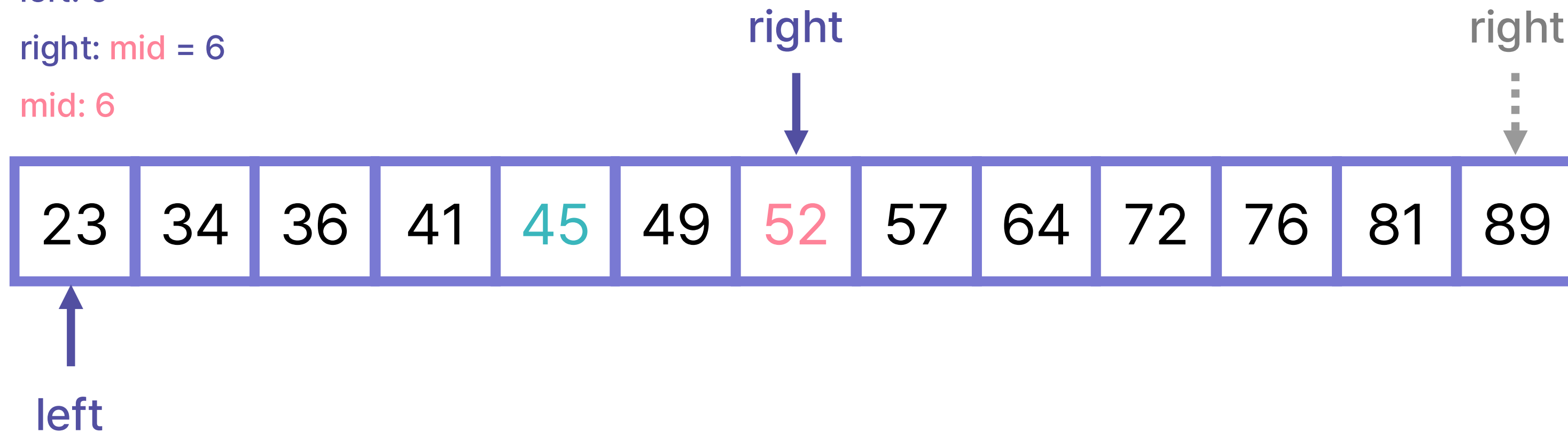
이진 탐색 (Binary Search)

target: 45

left: 0

right: mid = 6

mid: 6



처음 진행한 예시를 보면, mid가 우리가 찾는 값보다 크므로

right를 mid 자리로 옮겨줌

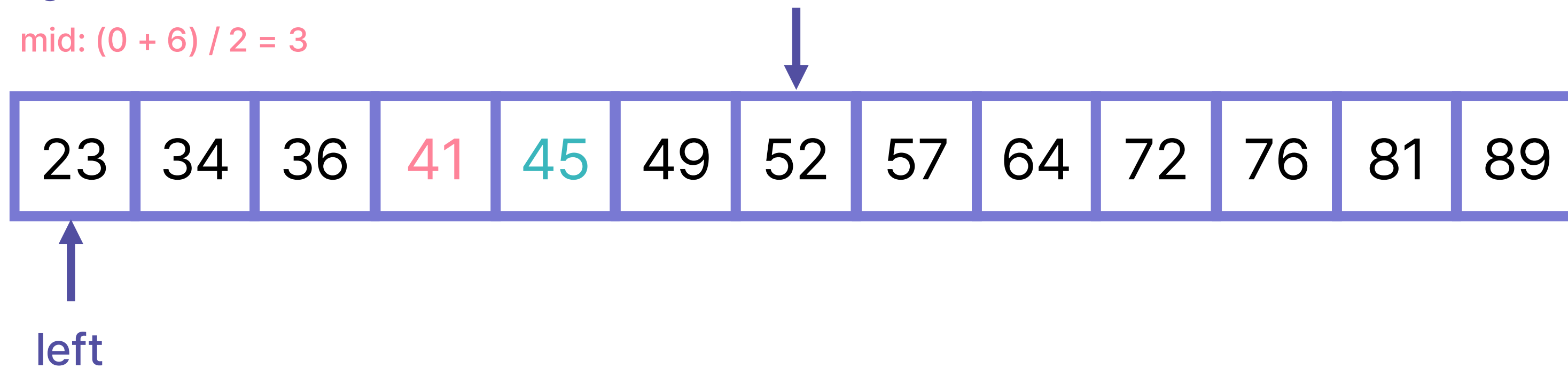
이진 탐색 (Binary Search)

target: 45

left: 0

right: 6

mid: $(0 + 6) / 2 = 3$



옮기고 난 후엔, 처음에 했던 것처럼

다시 left와 right를 기준으로 mid의 위치를 정함

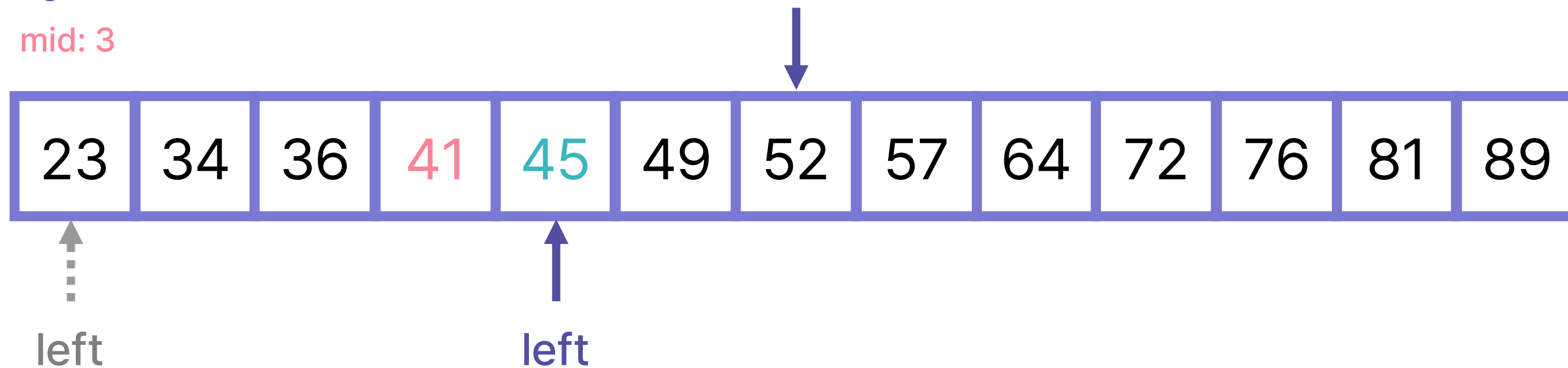
이진 탐색 (Binary Search)

target: 45

left: $\text{mid} + 1 = 4$

right: 6

mid: 3



이번에는 **mid**의 값이 찾는 값보다 작으므로,

left를 **mid** 다음 칸으로 옮겨줌

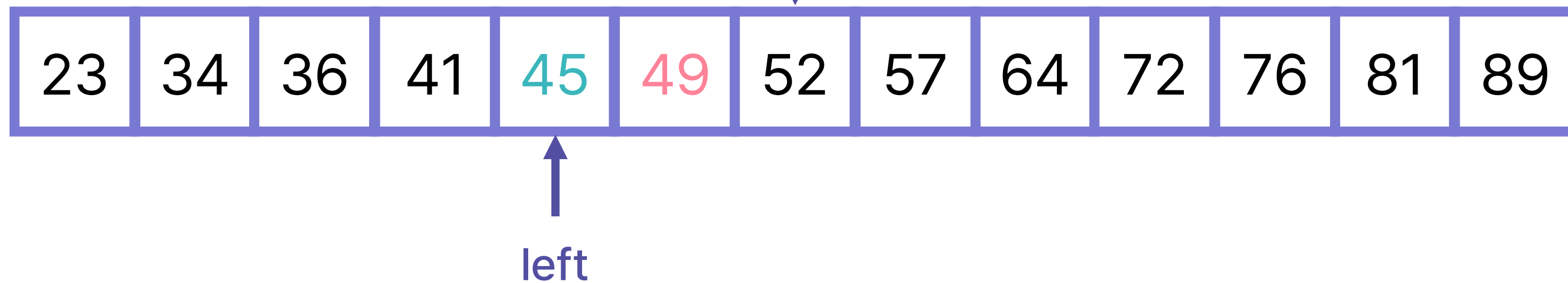
이진 탐색 (Binary Search)

target: 45

left: 4

right: 6

mid: $(4 + 6) / 2 = 5$



이 과정을 **left**와 **right**가 같은 칸을 가리킬 때까지 진행해야 함

다시 한번, **mid**를 구하면 49가 됨



이진 탐색 (Binary Search)

target: 45

left: 4

right: 6

mid: 5

23	34	36	41	45	49	52	57	64	72	76	81	89
----	----	----	----	----	----	----	----	----	----	----	----	----

right



left



이번에는 어떤 값이 이동해야 할까?

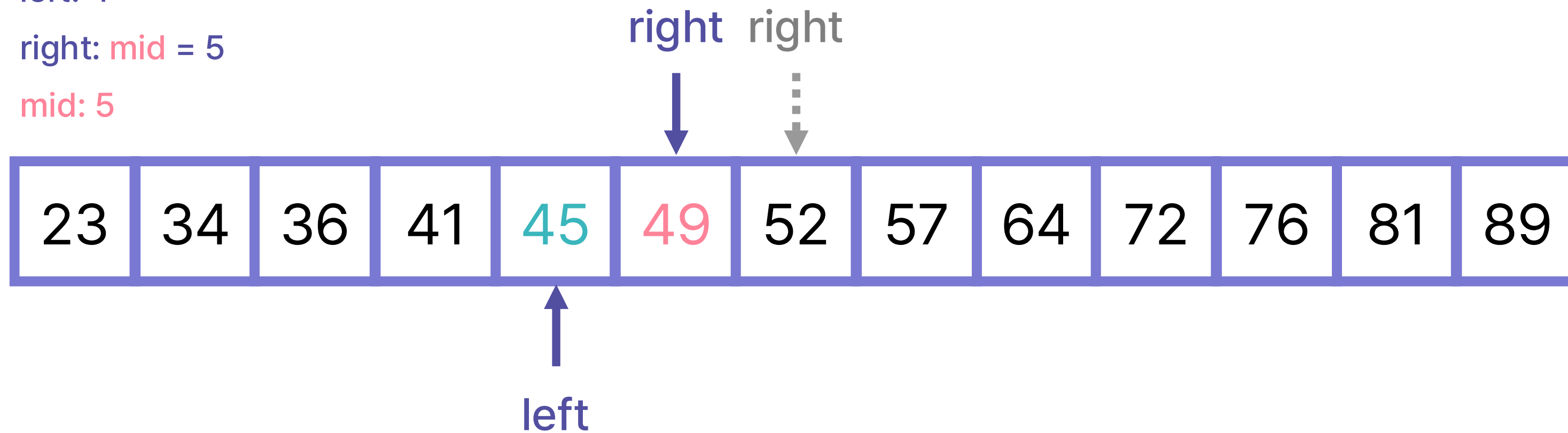
이진 탐색 (Binary Search)

target: 45

left: 4

right: mid = 5

mid: 5



mid 위치에 있는 값이 현재 값 보다 크기 때문에,
당연히 right가 이동해야 함



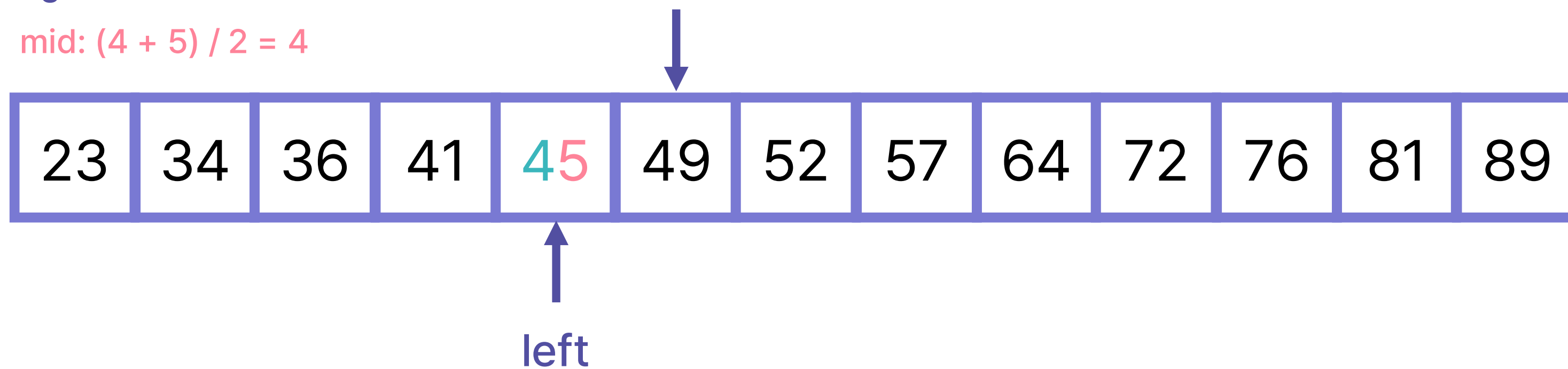
이진 탐색 (Binary Search)

target: 45

left: 4

right: 5

mid: $(4 + 5) / 2 = 4$



이번엔 mid 값이 찾는 값이 되었음



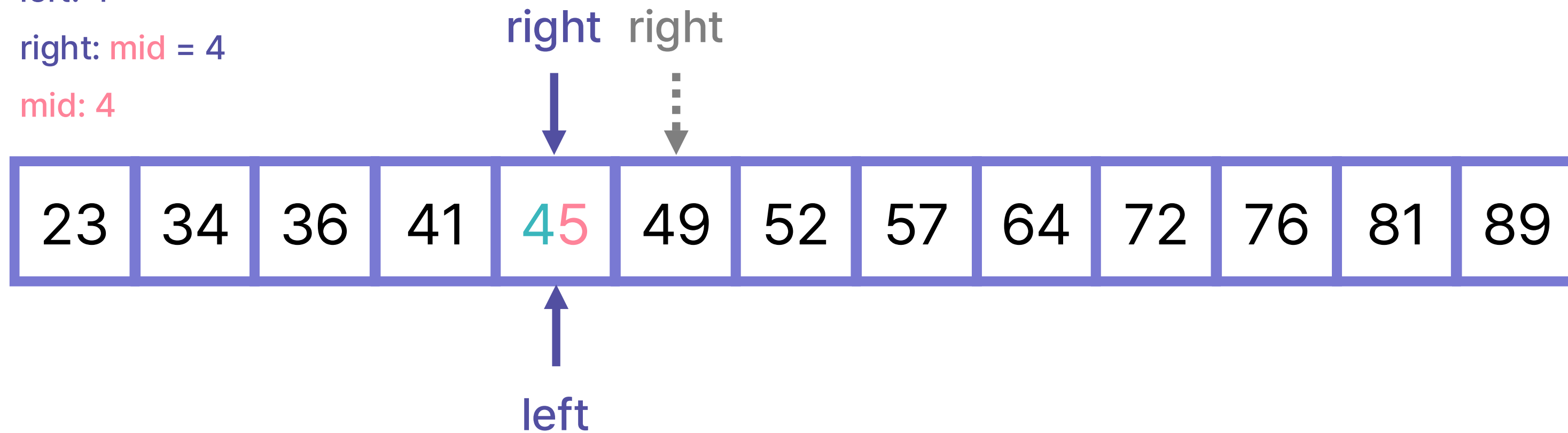
이진 탐색 (Binary Search)

target: 45

left: 4

right: mid = 4

mid: 4



mid 위치에 있는 값이 찾는 값과 동일하기 때문에,

right를 mid 위치로 옮겨줘야 함

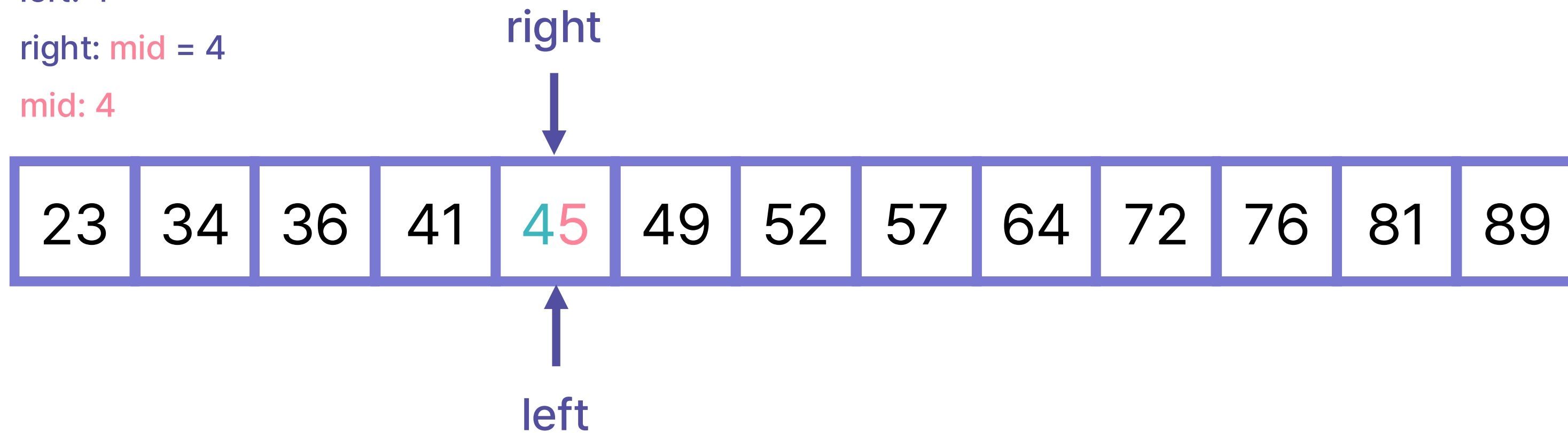
이진 탐색 (Binary Search)

target: 45

left: 4

right: mid = 4

mid: 4



이제 left와 right가 같은 위치를 가리키는 상태에서
mid 값이 target과 일치하여 이진 탐색 알고리즘이 종료됨



이진 탐색의 시간 복잡도

그렇다면, 실제로 얼마나 효율적일까?



left와 right가 정해지면,
mid는 두 지점의 **중간 위치**가 됨



이진 탐색의 시간 복잡도



이후, left나 right 중 하나가 mid 근처로 이동하게 되는데,
그 과정에서 left와 right가 포함하고 있는 범위가 **반으로 줄어들게 됨**



이진 탐색의 시간 복잡도



즉, 전체 범위를 계속 **반으로 줄여가면서**,
길이가 1이 되는 순간 이진 탐색 알고리즘이 끝나게 됨



이진 탐색의 시간 복잡도

$$64 = 2 \times 2 \times 2 \times 2 \times 2 \times 2$$

예를 들어, 배열의 길이가 64라면,
2로 6번 나누면 1이 되므로, 탐색을 6번만 진행해도 됨



이진 탐색의 시간 복잡도

$$6 = \log_2 64$$

즉, 이진 탐색은 **로그 시간 복잡도**를 보이는 알고리즘

순차 탐색의 시간 복잡도는 $O(N)$



이진 탐색의 시간 복잡도

$$4,294,967,296 = 2^{32}$$

순차 탐색 = 4,294,967,296회

이진 탐색 = 32회

데이터의 수가 작다면 잘 못 느낄 수 있지만,
데이터의 수가 많아지면 시간 측면에서 엄청난 차이를 보임



그러나 이진 탐색은 정렬되지 않은 배열에서는 할 수 없기 때문에,

만약 정렬되지 않은 배열이라면

정렬 과정에서 더 많은 시간이 걸릴 수 있음

따라서, 모든 상황에서 이진 탐색을 사용하지 말고,

상황에 따라 순차 탐색과 이진 탐색 중 효율적인 것을 사용해야 함



이진 탐색 (Binary Search)

이진 탐색 알고리즘은 다음과 같은 방식으로 진행됨

1. 배열의 양 끝을 **left**와 **right**로 지정
2. 두 위치의 가운데에 **mid**를 지정
3. **mid**와 찾는 값의 대소관계를 비교하여, **left**나 **right**를 이동
4. **left**와 **right**가 같은 지점에서 만나기 전 까지 1~3 을 반복



이진 탐색 (Binary Search)

- 결정 문제(Decision Problem)에서 답이 이분적일 때 사용할 수 있는 탐색 기법
 - 결정 문제 : 답이 Yes or No인 문제를 의미
- 예시) 1부터 100까지 중 28 이상이 처음 나오는 위치를 찾는다면?
 - 이러한 경우에 결정 문제는 $v[i] \geq 28$ 가 됨
 - 결정 문제의 답은 i 가 증가함에 따라 F, F, ..., F, T, T, ..., T 로 분포

F	F	F	F	F	F	T	T	T	T	T
---	---	---	---	---	---	---	---	---	---	---

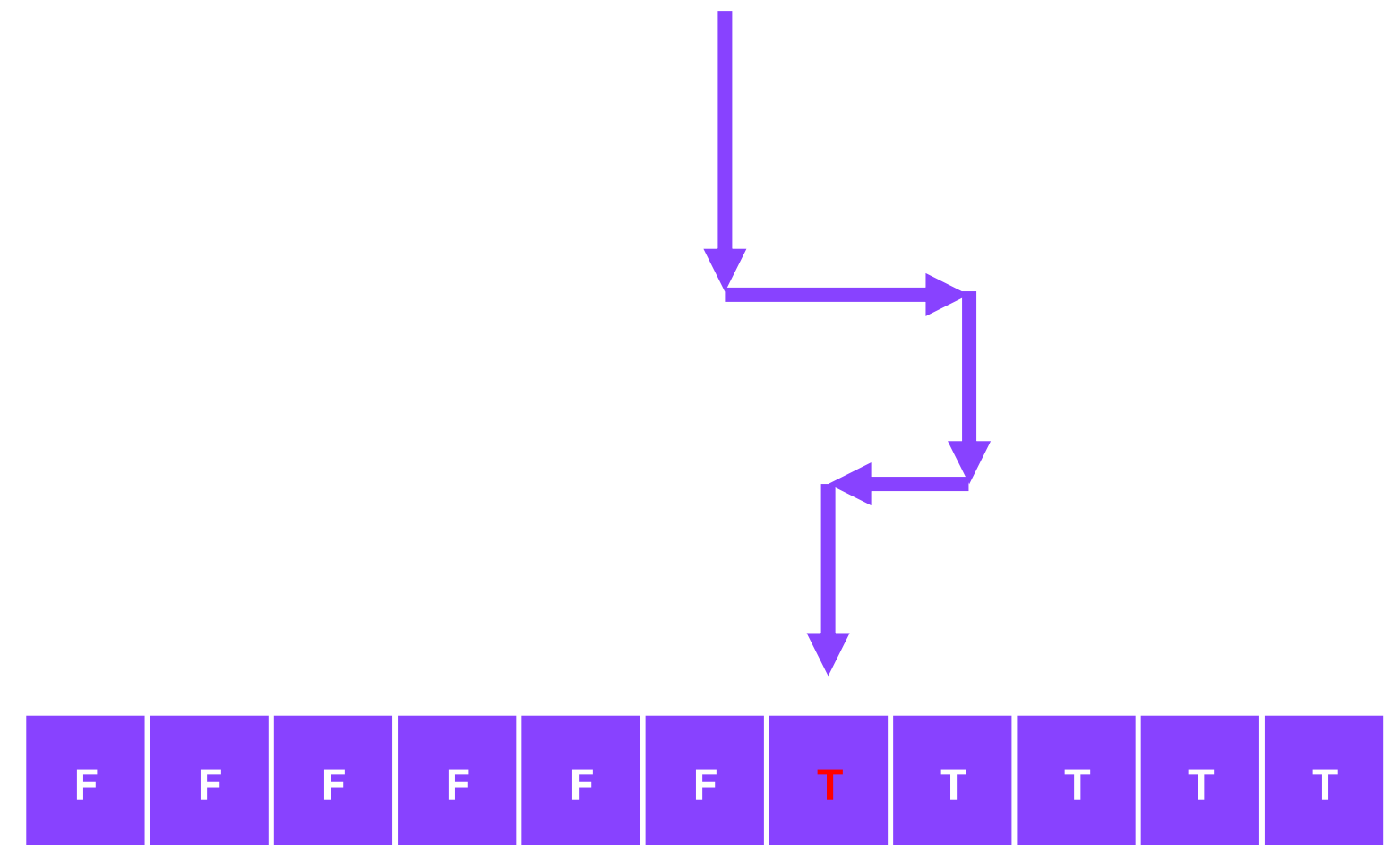


특정 숫자를 맞추는 Up & Down 게임



이진 탐색 (Binary Search)

- 이때, 결정 문제의 답이 처음 T가 되는 i 의 값을 찾으면 됨
 - 결정 문제의 답이 처음 T다 $\rightarrow$ T의 값이 28
- 결정 문제의 답이 두 구간으로 나뉘 $\rightarrow$ 이분적
- 이진 탐색을 이용하여 결정 문제의 답이 달라지는 경계를 찾으면 됨
 - $[1 \sim 100] \rightarrow [1 \sim 50] \rightarrow [25 \sim 50] \rightarrow \dots \rightarrow [27 \sim 28]$
 - 범위를 위와 같이 줄여 나가며 답을 구함





이진 탐색 (Binary Search)

- 이진 탐색(Binary Search)의 핵심 IDEA

- 경계를 포함하는 구간 $[lo, hi]$ 을 잡음
- 구간의 길이를 절반씩 줄여 나가며, lo 와 hi 가 경계에 위치하도록 함
- 이진 탐색이 끝난 뒤에 lo 의 다음 칸은 hi 다.
 - $lo + 1 = hi$, $Check(lo) \neq Check(hi)$
 - $Check(x) \rightarrow$ 결정 문제의 parameter가 x 일 때의 결정 문제의 답
 - Check 함수 내부에서 답이 T인지 F인지 판단
 - 이진 탐색에서는 보통 하나의 parameter를 가짐

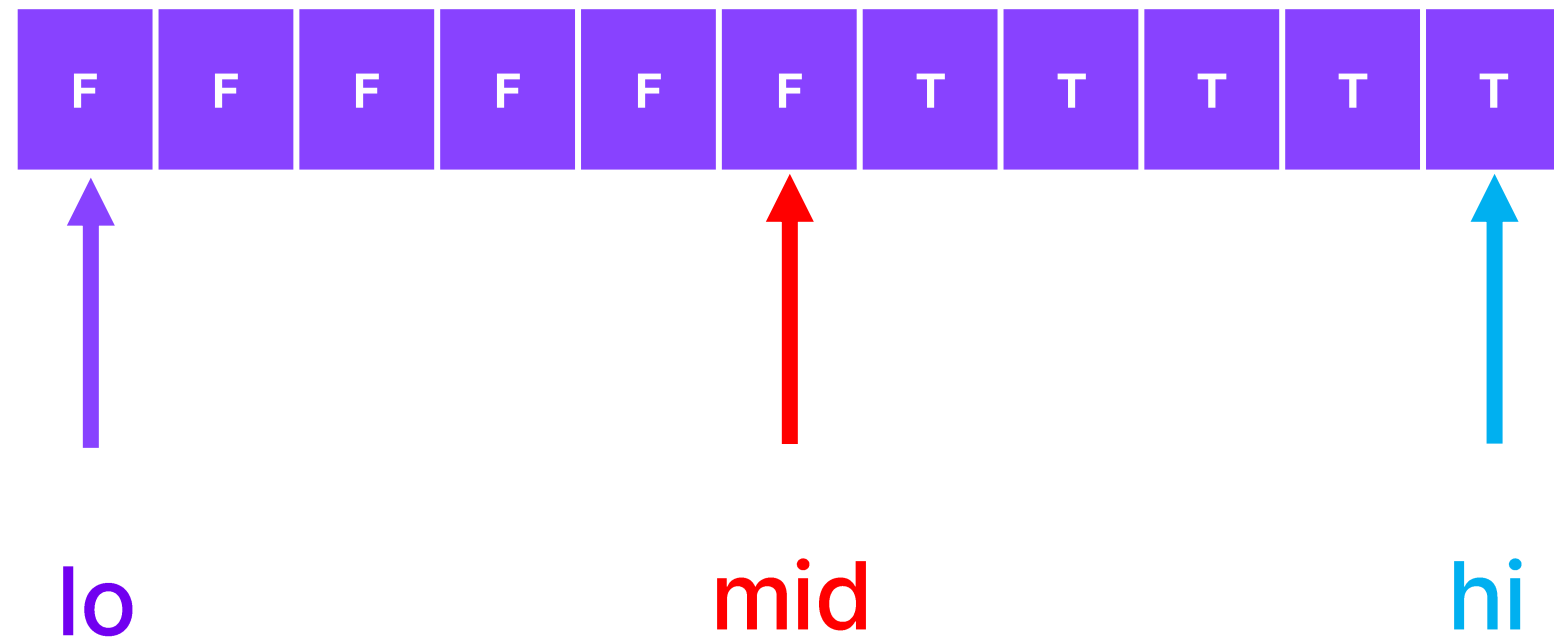
- 구간의 범위가 클 때, 효과적으로 원하는 값을 찾을 수 있음

- 시간 복잡도 : $O(\log N)$

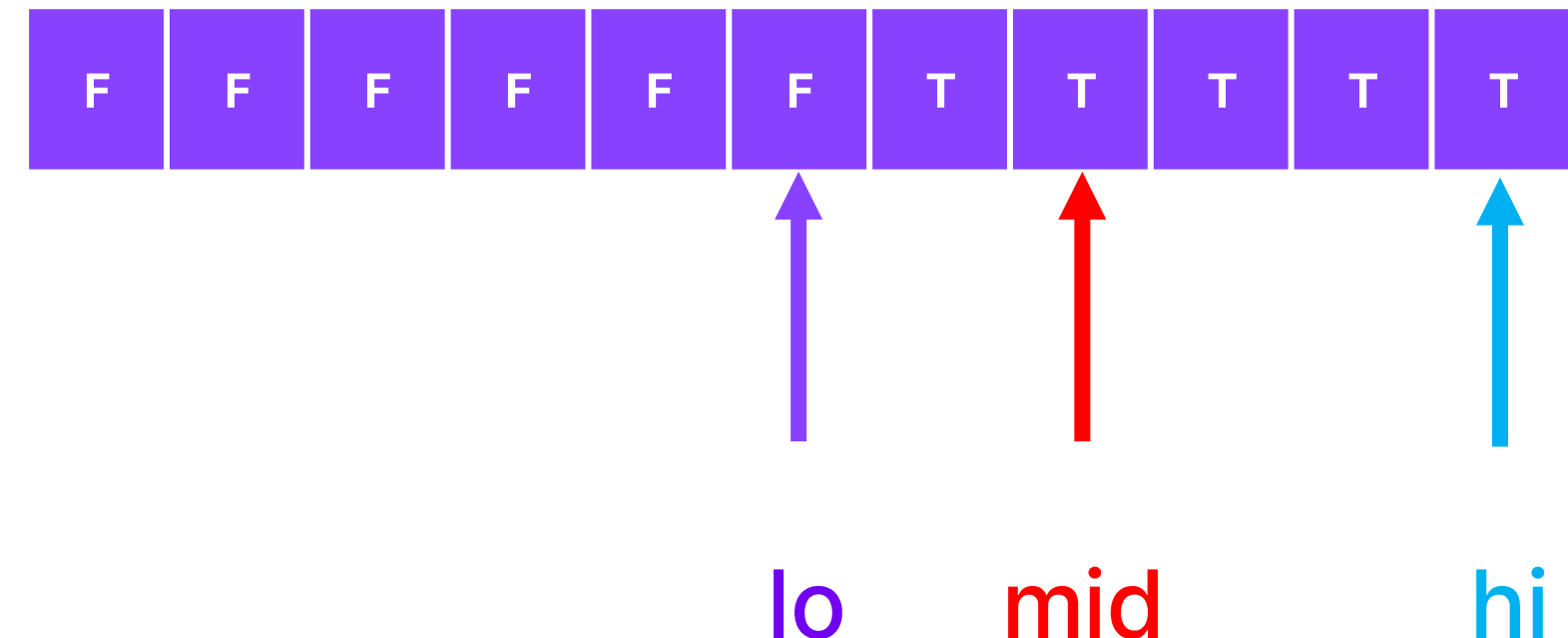
```
# 답이 [a, b] 구간 내에  
존재한다고 할 때  
lo, hi = a - 1, b  
  
while lo + 1 < hi :  
    mid = (lo + hi) // 2  
    if not check(mid):  
        lo = mid  
    else:  
        hi = mid
```



이진 탐색의 구현



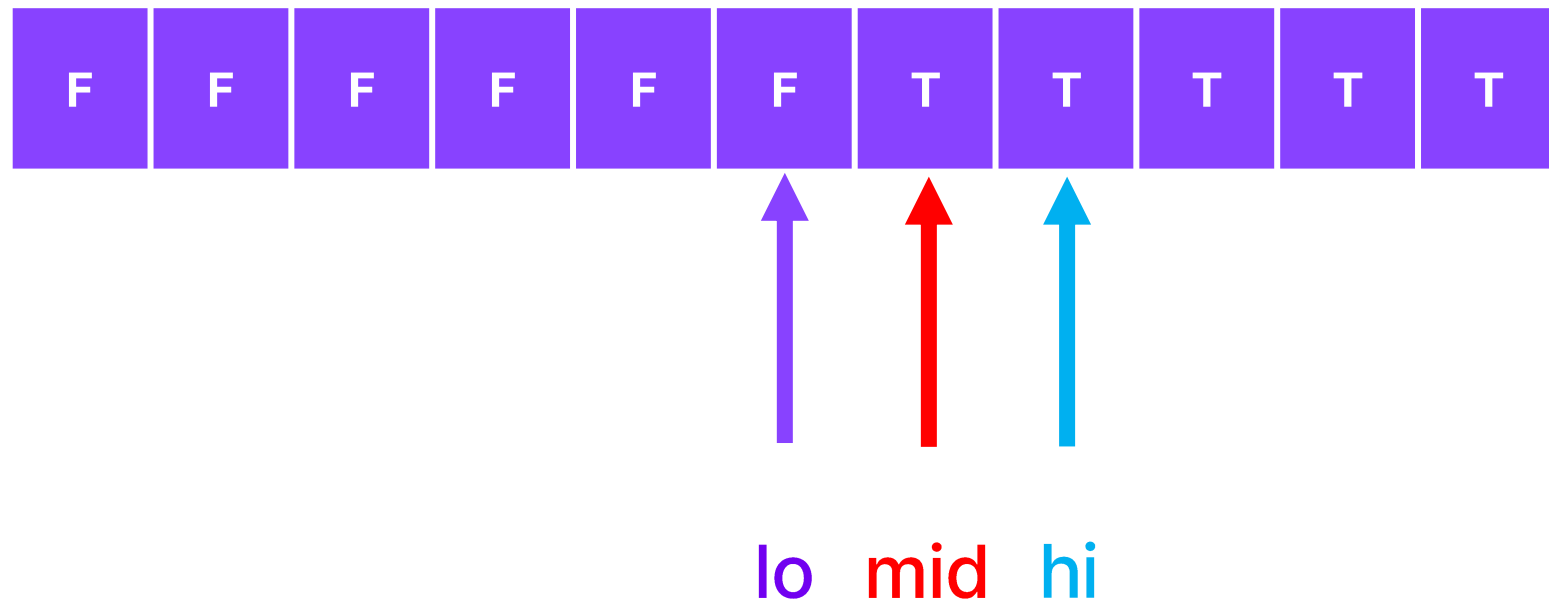
경계를 포함하면서 $\text{Check}(\text{lo}) \neq \text{Check}(\text{hi})$ 가 되도록 $[\text{lo}, \text{hi}]$ 를 잡음



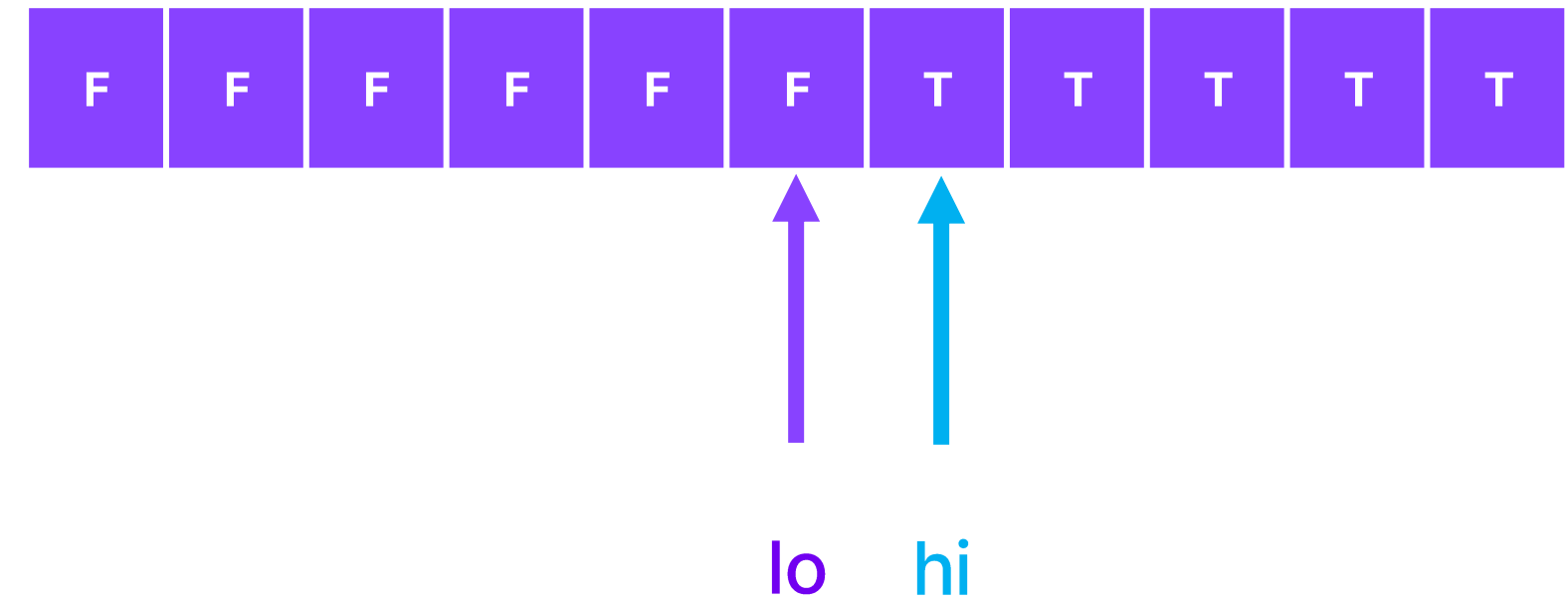
$\text{Check}(\text{lo}) == \text{mid}$ 라면 $\text{lo} = \text{mid}$,
아니라면 $\text{hi} = \text{mid}$ 를 반복



이진 탐색의 구현



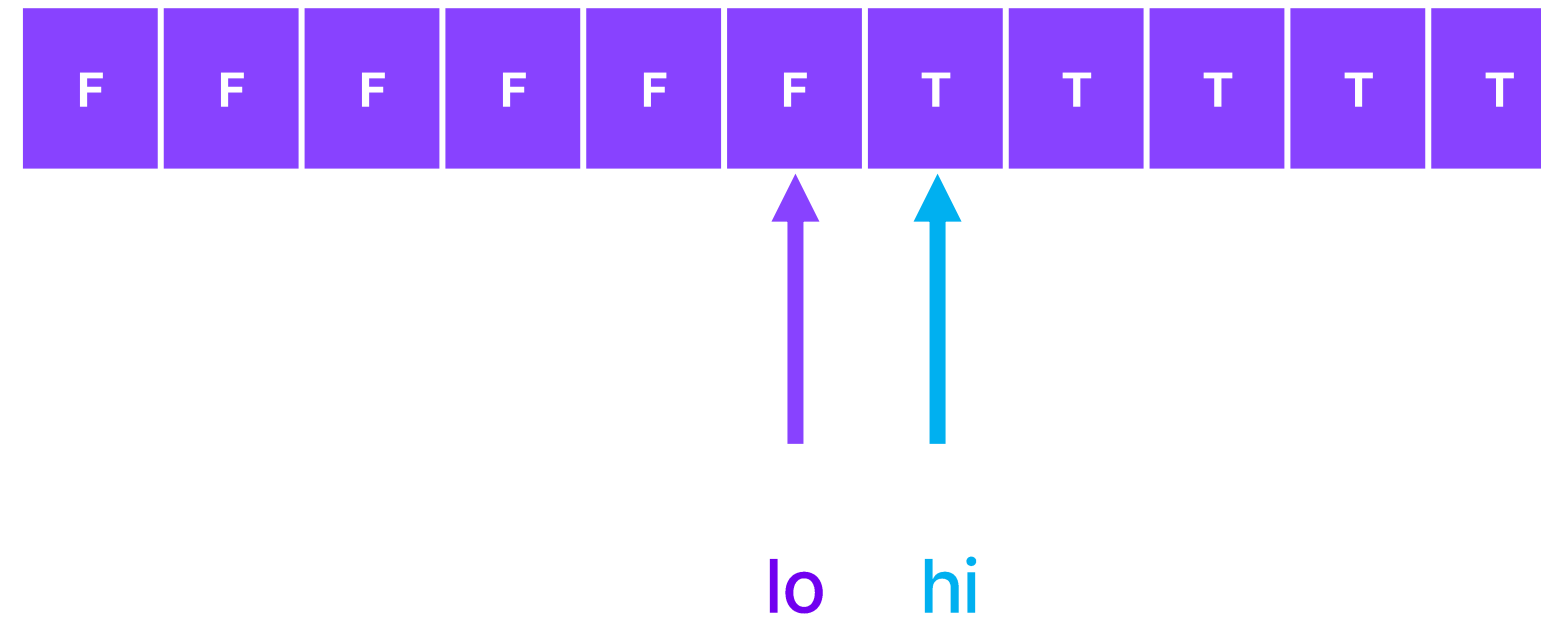
Check(lo) == mid라면 lo = mid,
아니라면 hi = mid를 반복



lo + 1 == hi가 되면 탈출,
lo, hi는 경계에 위치



이진 탐색의 구현



이진 탐색이 끝나면 **lo**, **hi**는 결정 문제의 답이 바뀌는 경계에 위치하게 되며, 만약 결정 문제의 답의 분포가 F~T인데 정답이 가장 큰 F라면 **lo**, 가장 작은 T라면 **hi**를 출력



이진 탐색의 구현

이진 탐색의 구현은 크게 3가지로 나눌 수 있음

1. 경계를 포함하도록($\text{Check}(\text{lo}) \neq \text{Check}(\text{hi})$ 가 되도록) $[\text{lo}, \text{hi}]$ 설정
 - 초기에 $\text{Check}(\text{lo}) \neq \text{Check}(\text{hi})$ 이기 때문에, 결정 문제의 답이 바뀌는 경계는 $[\text{lo}, \text{hi}]$ 내에 있음이 보장
2. $\text{lo} + 1 < \text{hi}$ 인 동안, $\text{Check}(\text{lo}) == \text{mid}$ 라면, $\text{lo} = \text{mid}$, 아니라면 $\text{hi} = \text{mid}$ 를 반복
 - 반복되면서 $[\text{lo}, \text{hi}]$ 가 $[\text{lo}, \text{mid}]$ 혹은 $[\text{mid}, \text{hi}]$ 가 됨
3. $\text{lo} + 1 == \text{hi}$ 가 되면 탈출, lo 와 hi 는 경계에 위치
 - 이진 탐색이 끝나면 lo , hi 는 결정 문제의 답이 바뀌는 경계에 위치하게 되며, 만약 결정 문제의 답의 분포가 F~T인데 정답이 가장 큰 F라면 lo , 가장 작은 T라면 hi 를 출력
 - 단, 문제에 따라서 결정 문제의 분포가 F~T가 아닌 T~F일 수도 있음
 - 이때 정답은 경계의 왼쪽(lo)일 수도 있고, 오른쪽(hi)일 수도 있음

파라메트릭 서치 (Parametric Search)

- Parametric Search : 최적화 문제를 결정 문제(Yes or No)로 바꾸어 해결하는 기법
 - 최적화 문제 : a 에서 b 로 가는 최단거리를 구하여라(문제의 답에 해당하는 최적값을 구하는 문제)
 - 결정 문제 : a 에서 b 로 가는 거리가 x 이하인 경로가 있는가?(Yes or No로 답변할 수 있는 문제)
- 최적화 문제는 주어진 값 중에서 조건을 만족하는 최대/최소값을 구하는 문제
 - 이때, 조건은 Yes or No로 대답할 수 있는 문제
 - 최적화 문제의 답은 조건이 Yes(True)인 값 중 가장 크거나 가장 작은 값이 됨
- 탐색 범위가 큰 경우, 시간 복잡도 $O(n)$ 의 방법으로는 시간 초과가 발생할 때 사용

파라메트릭 서치 (Parametric Search)

- 최적화 문제가 아래의 조건을 만족하면 결정 문제로 바꿨을 때, 답을 구하기 더 쉬워짐
 - 답이 가질 수 있는 값의 범위에 대해, 답이 만족해야 하는 조건 C 를 만족하는 구간과 만족하지 않는 구간 두 개가 모두 연속
 - 답을 고정하였을 때, 조건 C 를 확인하는 게 어렵지 않음
- 조건 C 를 확인하는 시간 복잡도가 $f(x)$ 이고, 답이 가질 수 있는 값의 범위가 N 이면, 이진 탐색을 이용하여 $O(f(x) \times \log N)$ 에 최적값을 구할 수 있음

파라메트릭 서치 (Parametric Search)

- 주어진 최적화 문제의 정답이 x 이고, x 가 가질 수 있는 값의 범위를 수직선으로 표현($A \sim B$)



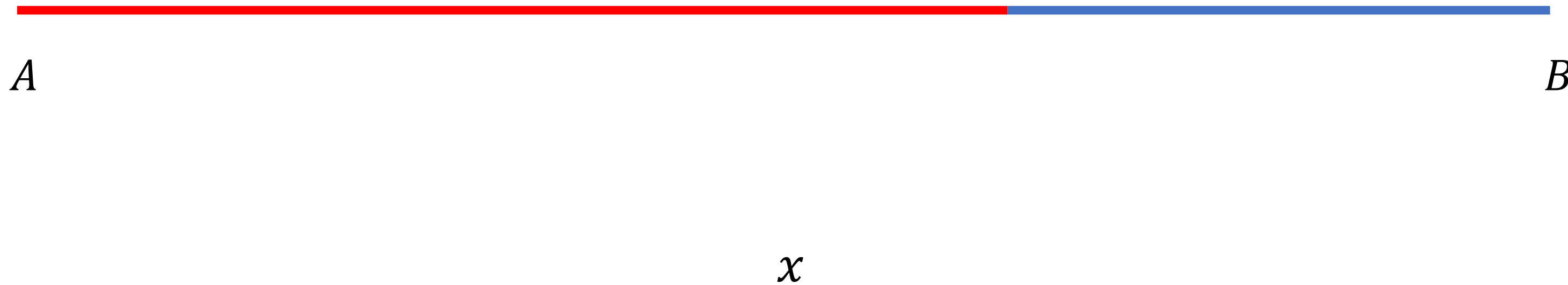
파라메트릭 서치 (Parametric Search)

- 주어진 최적화 문제의 정답이 x 이고, x 가 가질 수 있는 값의 범위를 수직선으로 표현($A \sim B$)
- 조건 C 에 대해 **C 를 만족하는 연속한 구간**과 **그렇지 않은 구간** 두 개로 구성된다면, 문제를 결정 문제로 바꾸어 이진 탐색을 시도할 수 있음



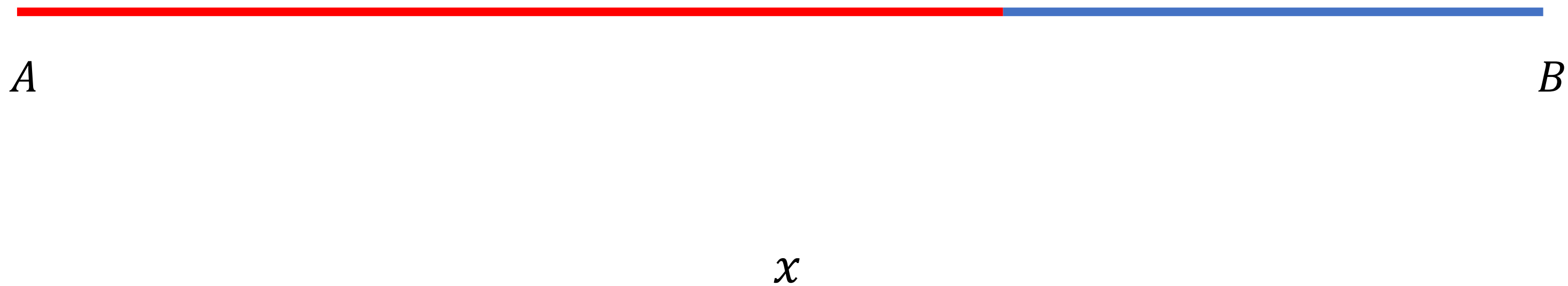
파라메트릭 서치 (Parametric Search)

- A 와 B 의 중간값 x
- x 는 조건 C 를 만족한다.



파라메트릭 서치 (Parametric Search)

- A 와 B 의 중간값 x
- 이때, 조건 C 를 만족하는 구간은 항상 연속
 - x 보다 작은, x 보다 왼쪽의 모든 값은 조건 C 를 만족함이 보장 $\rightarrow$ 오른쪽 절반 구간만 확인하면 됨



파라메트릭 서치 (Parametric Search)

- 이제 다시 오른쪽 구간의 절반이 되는 위치를 x
- 이때의 x 의 위치는 조건 C 를 만족하지 않음
 - 이 위치보다 오른쪽의 구간은 탐색할 필요가 없음





예시 - 나무 자르기

- 나무 M 미터가 필요하여, 벌목 전의 나무 N 개와 목재 절단기를 이용하여 나무를 얻으려 한다
- 절단기 높이를 H 로 조정 $\rightarrow H$ 보다 높은 나무는 잘리고, H 보다 낮은 나무는 잘리지 않는다
- 만약, 나무의 높이가 $[10, 20, 30, 40, 50]$ 이고 H 가 30이라면, 절단 후 나무들의 높이는 $[10, 20, 30, 30, 30] \rightarrow$ 총 나무 30미터를 얻는다 (잘린 10미터 나무와 20미터 나무의 합)
- 총 나무 M 미터를 얻기 위해 설정할 수 있는 H 의 최댓값을 구하여라
- 이때, 각 나무들의 높이는 1이상 10^9 이하의 정수다 ($1 \leq N \leq 10^5, 1 \leq M \leq 2 \times 10^9$)



예시 - 나무 자르기

- 가능한 H 의 최댓값을 구해야 하므로, 이 문제는 최적화 문제라고 볼 수 있음
- 최적화 문제를 결정 문제로 바꿔보자.
 - 조건 : H 의 값을 K 로 정할 때, 엘리스 토끼가 얻을 수 있는 나무의 높이는 M 이상인가?
 - 이 조건은 만족하는 구간과 만족하지 않는 구간이 연속한 두 개로 나뉨
 - 조건을 만족하는 K 의 최댓값을 구하면 됨
- 이러한 유형의 문제를 풀 때 가장 신경 써야 할 점은 조건을 잘 정하는 것
- 찾아야 하는 가능한 답의 범위가 많이 넓은 경우에 Parametric Search를 시도



이진 탐색 구현 시, 자주하는 실수들

- lo , hi 는 항상 정답의 범위를 나타낼 수 있도록 해야 함
 - 예시) lo 를 출력해야 하는 문제의 답이 최대 n 일 때 $hi = n$ 으로 선언하거나, hi 를 출력해야 하는 문제의 답이 최소 0일 때 $lo = 0$ 으로 선언하면 안 됨($hi = n + 1$, $lo = -1$ 로 선언해야 함)
- Overflow에 주의
 - 이진 탐색을 사용하는 문제는 대부분 수의 범위가 크기 때문에 Overflow가 발생할 가능성이 있음
- 결정 문제의 정의에 맞게 `check` 함수를 잘 구현해야 함
 - 예시) `lower_bound`는 $v[i] \geq x$ 인 i 의 최솟값을 구하는 함수이고, `upper_bound`는 $v[i] > x$ 인 i 의 최솟값을 구하는 함수인데, `Check` 함수의 부등호를 조금만 틀려도 전혀 엉뚱한 값이 튀어나올 수 있음



이진 탐색 – 내부 라이브러리 함수 사용

```
import bisect

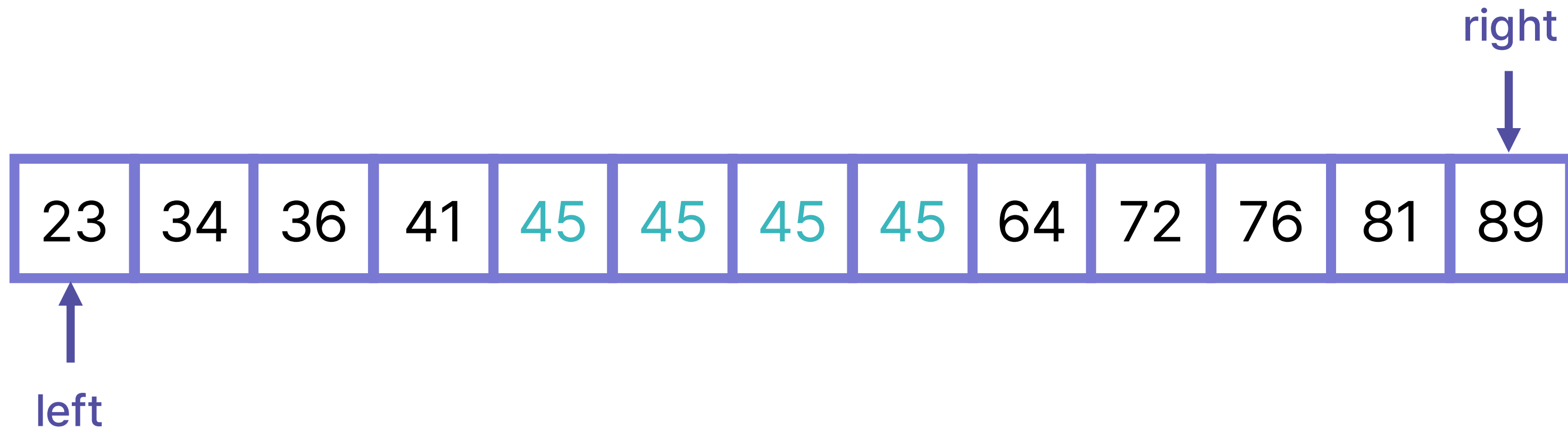
arr = [0, 1, 2, 3, 4, 7, 9, 10]
target = 3
print(bisect.bisect_left(arr, target)) # 3
print(bisect.bisect_right(arr, target)) # 4
print(bisect.bisect_left(arr, 6)) # 5, bisect_right를 사용했을 때도 동일
```

- Python에서는 배열 안의 특정 값의 존재 유무를 로그 시간으로 찾을 수 있는 함수는 없음
 - 단, 오름차순이 유지되도록 삽입하기 위한 인덱스를 찾는 함수는 존재
 - *import bisect* 필요
 - arr 안에 target이 있는 경우
 - *bisect_left(arr, target)* → 삽입할 수 있는 위치의 가장 왼쪽 인덱스 반환
 - *bisect_right(arr, target)* → 삽입할 수 있는 위치의 가장 오른쪽 인덱스 반환
 - arr 안에 target이 없는 경우
 - *bisect_left, bisect_right* → 삽입 위치를 반환

23	34	36	41	45	45	45	45	64	72	76	81	89
----	----	----	----	----	----	----	----	----	----	----	----	----

만약, 배열에 같은 값이 여러 개 있다면
이진 탐색의 결과는 어디를 가리킬까?

lower/upper bound



이진 탐색 알고리즘의 절차를 떠올리면서,
어디를 가리킬지 생각해보자

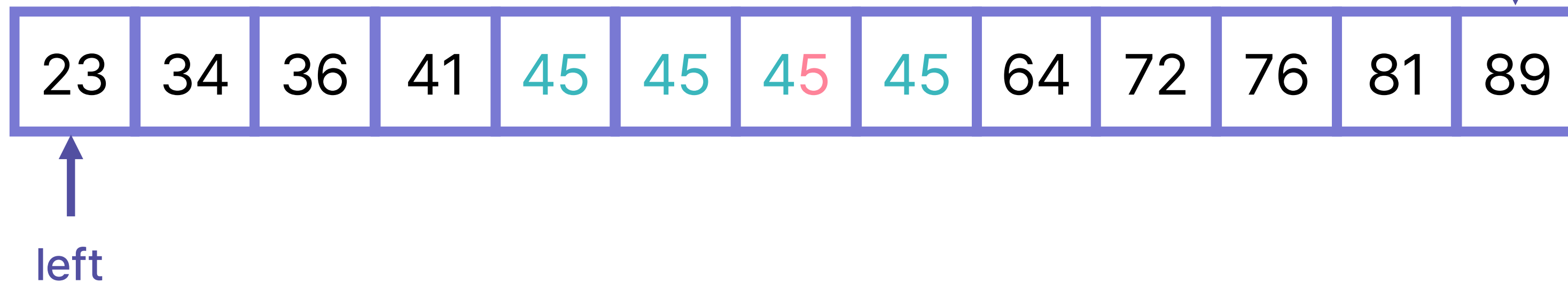
lower/upper bound

target: 45

left: 0

right: 12

mid: $(0 + 12) / 2 = 6$



이진 탐색 알고리즘의 절차를 떠올리면서,
어디를 가리킬지 생각해보자

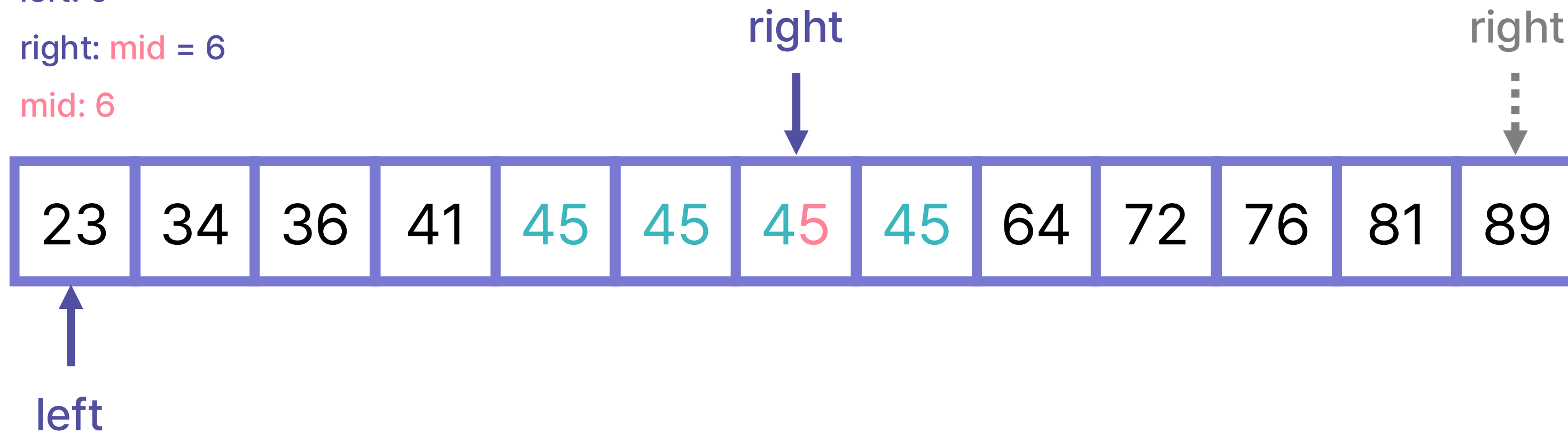
lower/upper bound

target: 45

left: 0

right: mid = 6

mid: 6



이진 탐색 알고리즘의 절차를 떠올리면서,
어디를 가리킬지 생각해보자

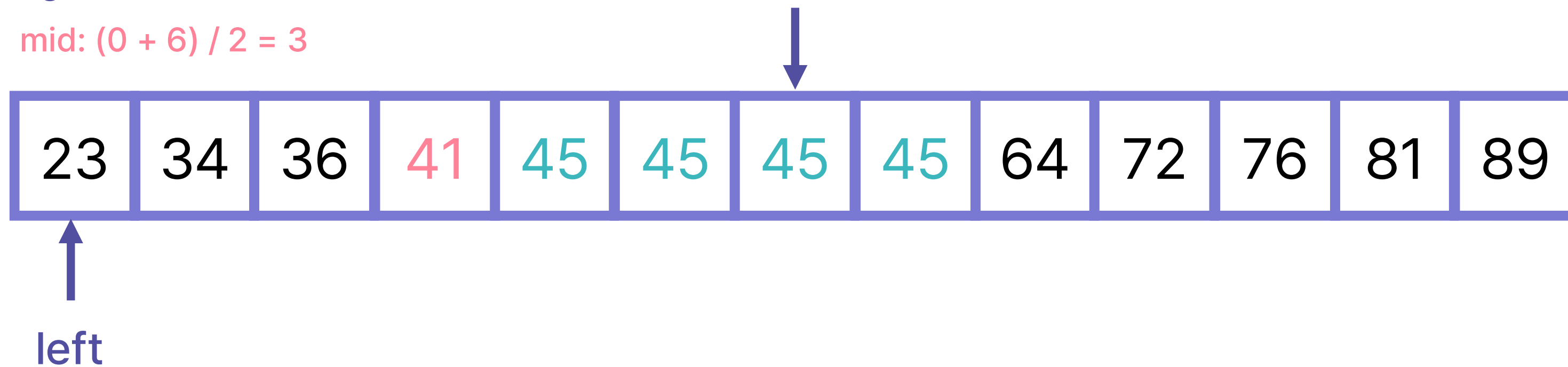
lower/upper bound

target: 45

left: 0

right: 6

mid: $(0 + 6) / 2 = 3$



이진 탐색 알고리즘의 절차를 떠올리면서,
어디를 가리킬지 생각해보자

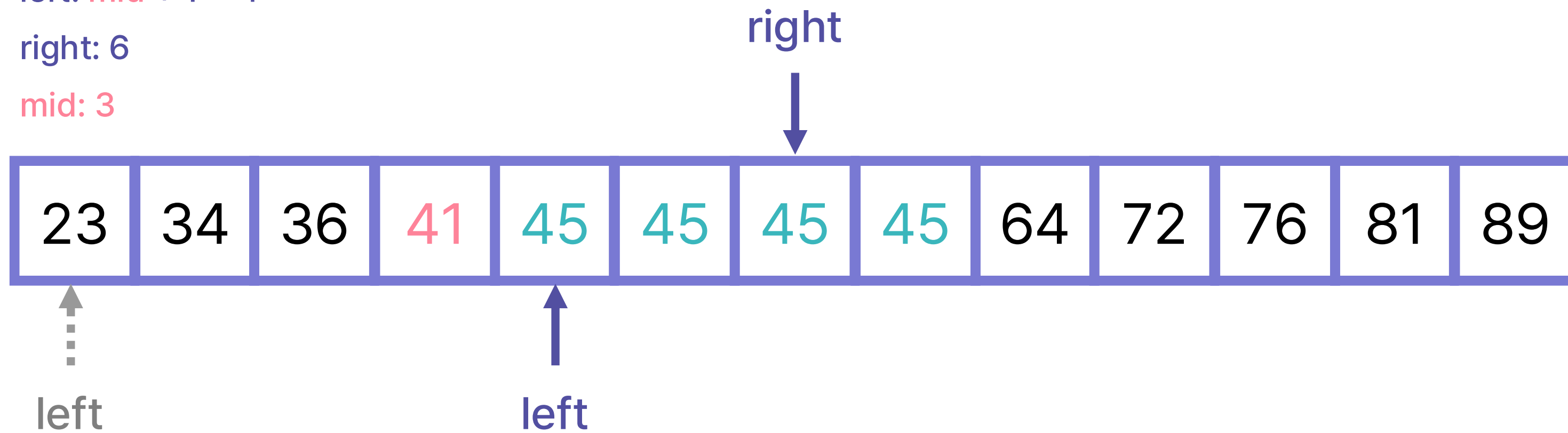
lower/upper bound

target: 45

left: $\text{mid} + 1 = 4$

right: 6

mid: 3



이진 탐색 알고리즘의 절차를 떠올리면서,
어디를 가리킬지 생각해보자

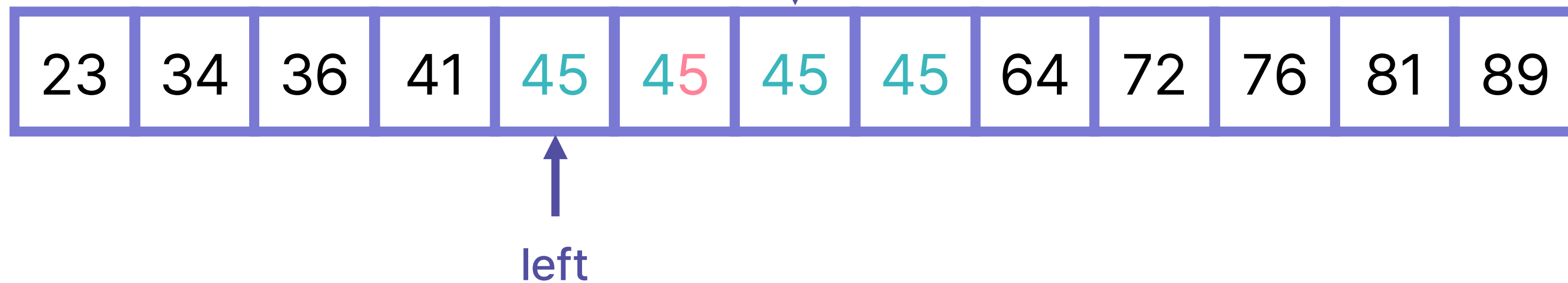
lower/upper bound

target: 45

left: 4

right: 6

mid: $(4 + 6) / 2 = 5$



이진 탐색 알고리즘의 절차를 떠올리면서,
어디를 가리킬지 생각해보자

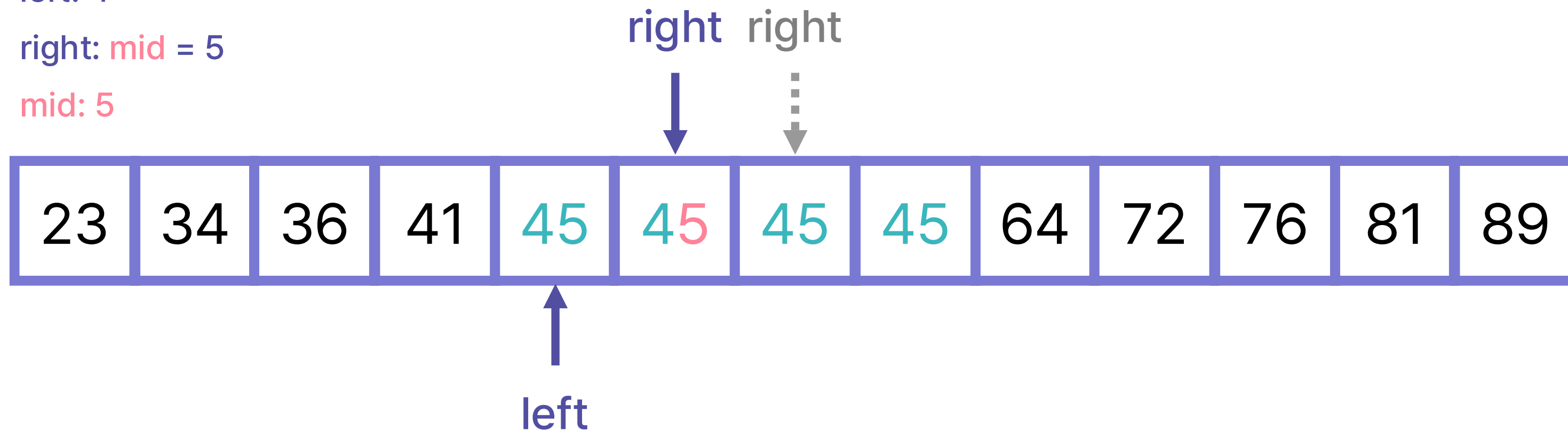
lower/upper bound

target: 45

left: 4

right: mid = 5

mid: 5



이진 탐색 알고리즘의 절차를 떠올리면서,
어디를 가리킬지 생각해보자

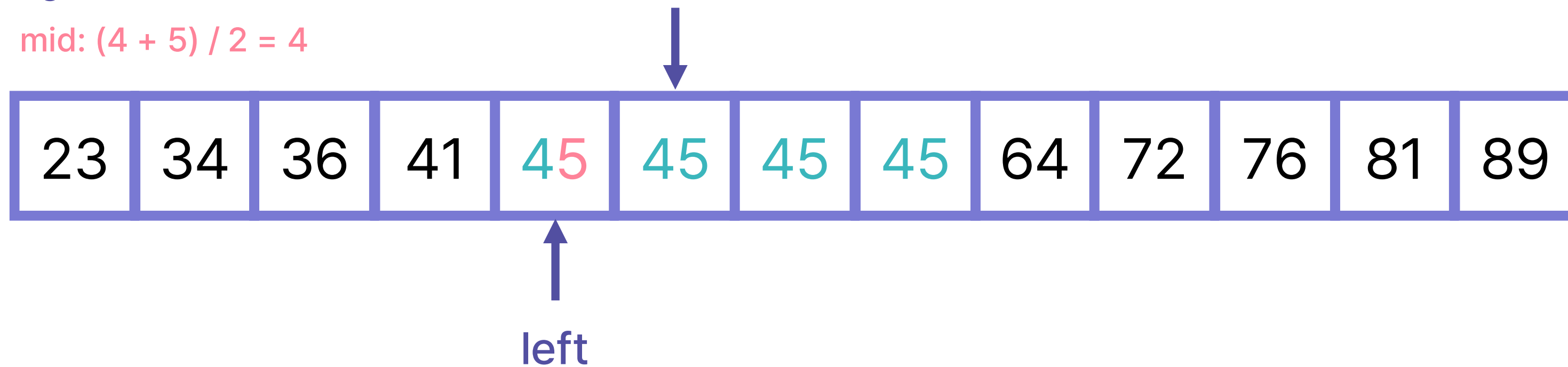
lower/upper bound

target: 45

left: 4

right: 5

mid: $(4 + 5) / 2 = 4$



이진 탐색 알고리즘의 절차를 떠올리면서,
어디를 가리킬지 생각해보자

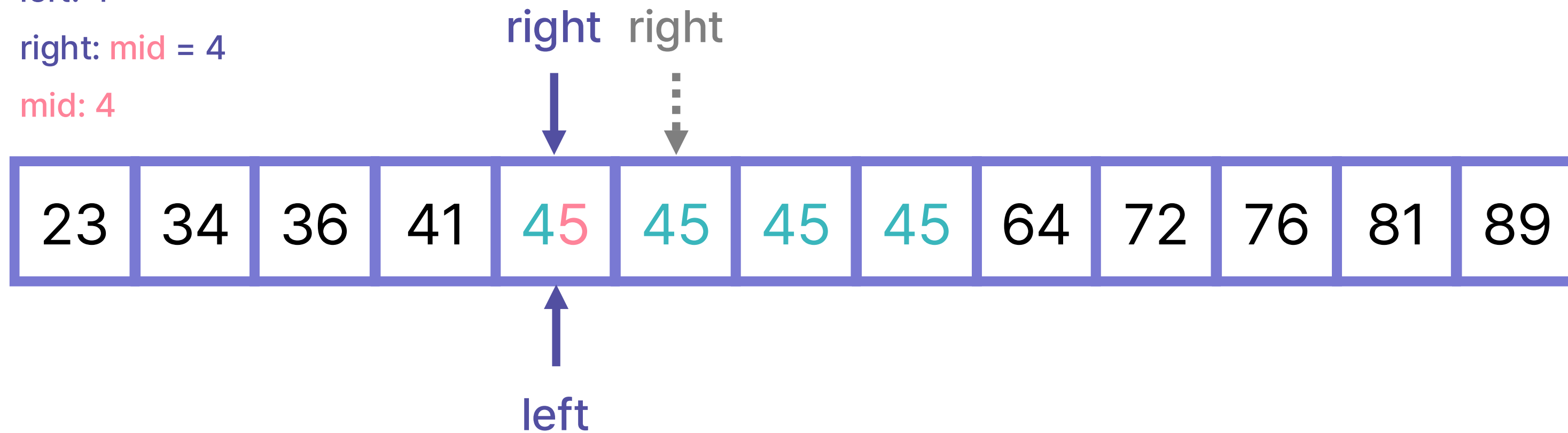
lower/upper bound

target: 45

left: 4

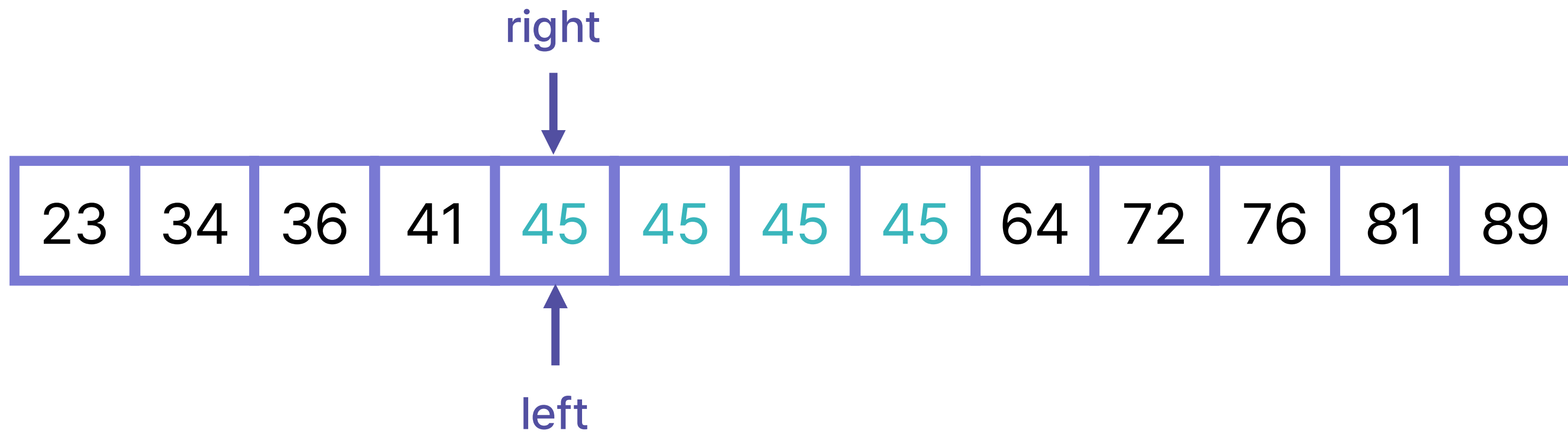
right: mid = 4

mid: 4

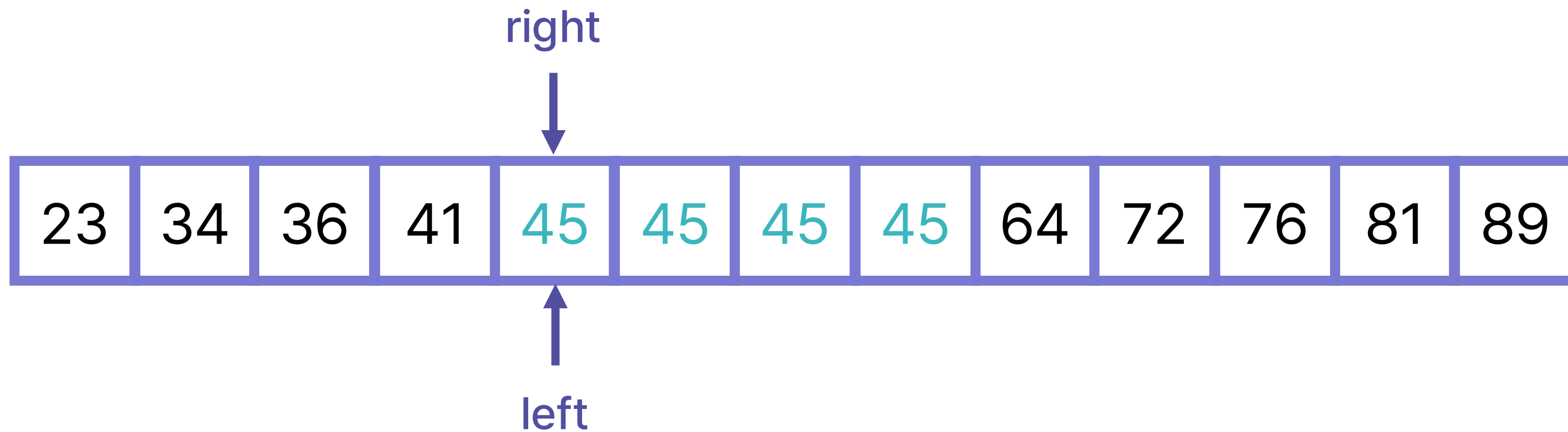


이진 탐색 알고리즘의 절차를 떠올리면서,
어디를 가리킬지 생각해보자

lower/upper bound

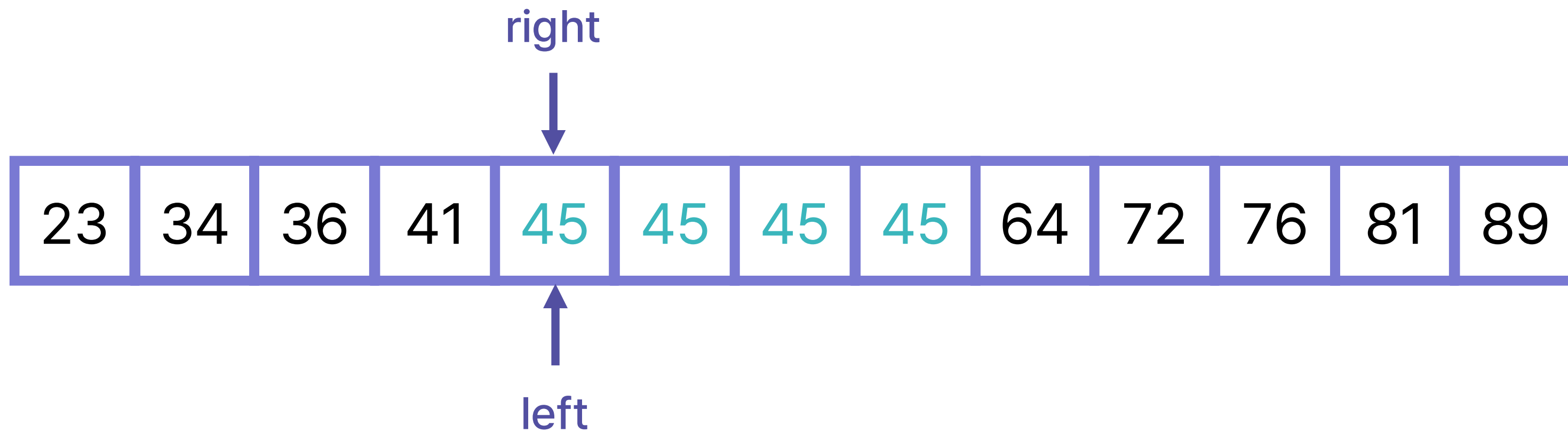


최종적으로 가리키는 곳은 동일한 숫자의 시작점이 됨



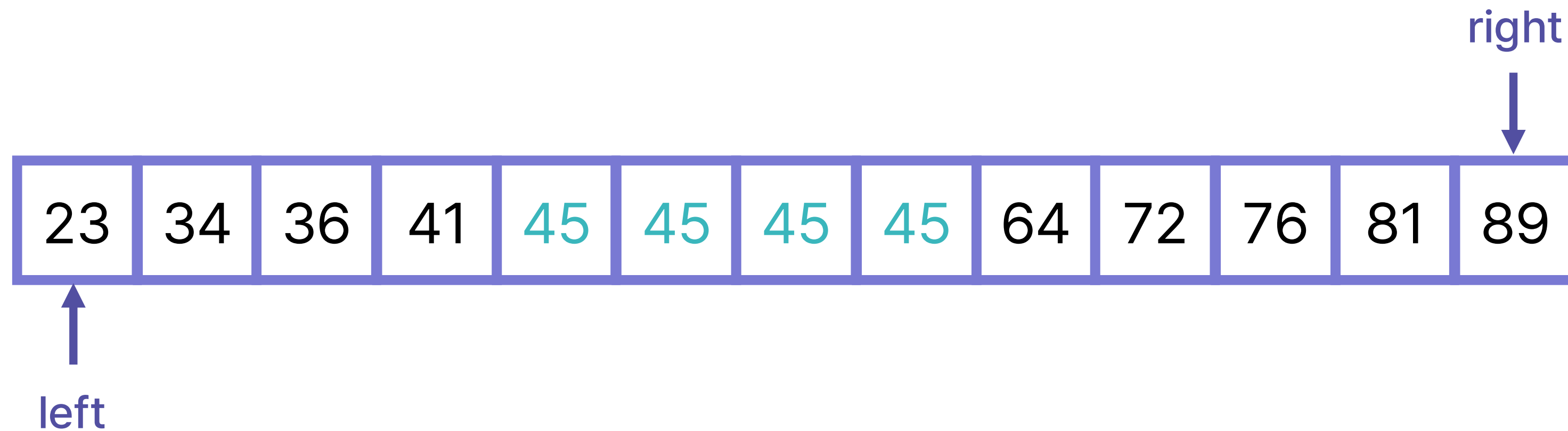
따라서, 동일한 숫자가 몇 개 이던 상관없이,
기존의 이진 탐색 알고리즘은 반드시 첫 번째 값을 가리킴

lower/upper bound



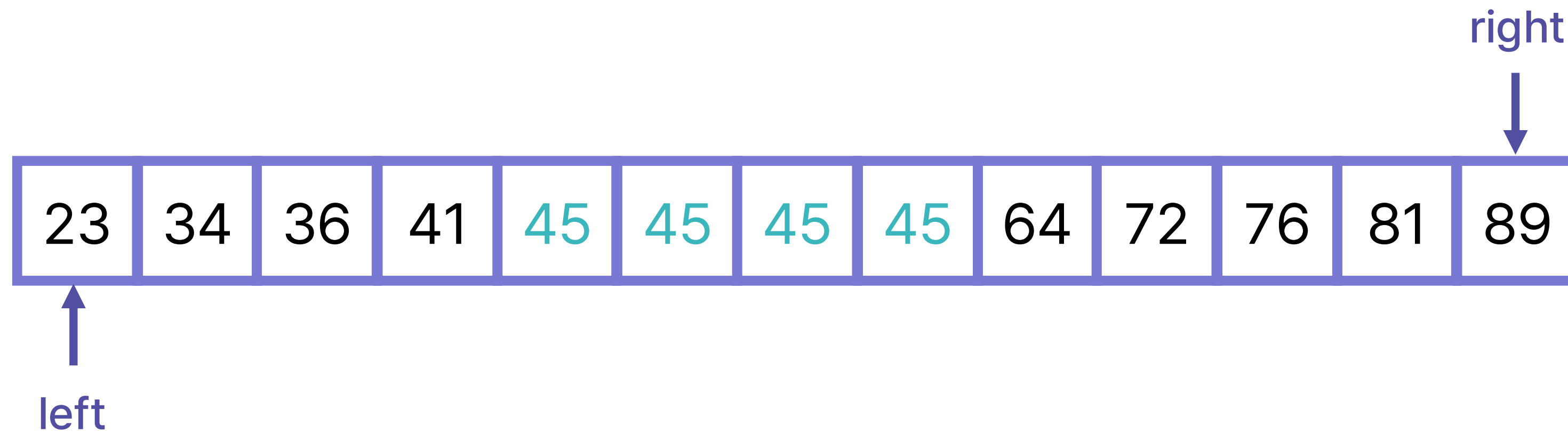
그렇다면, 배열의 나머지 값을 확인할 방법은 없을까?

lower/upper bound



기존 이진 탐색 알고리즘은 **mid** 값이
찾는 값보다 작을 때만 **left**를 움직였음

lower/upper bound



만약 찾는 값보다 작거나 **같을 때**
left를 움직인다면 어떤 결과가 나올까?

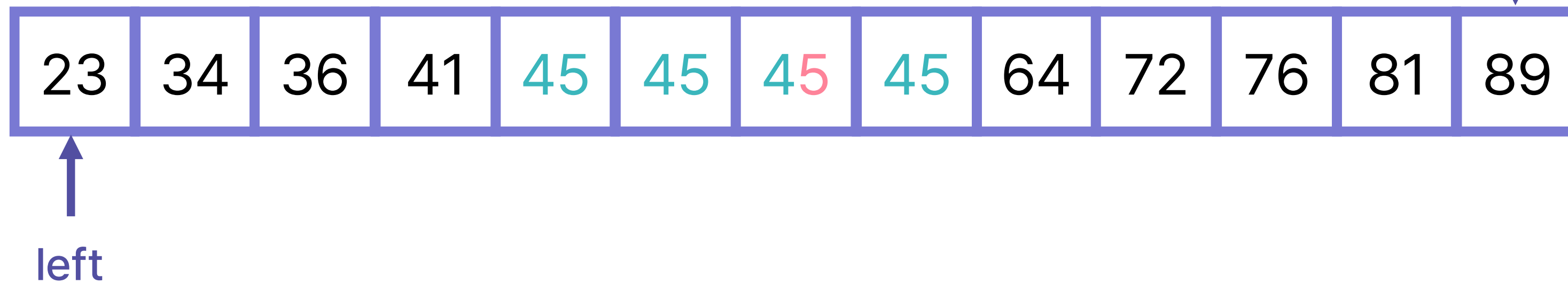
lower/upper bound

target: 45

left: 0

right: 12

mid: $(0 + 12) / 2 = 6$



먼저, 동일하게 mid 를 설정

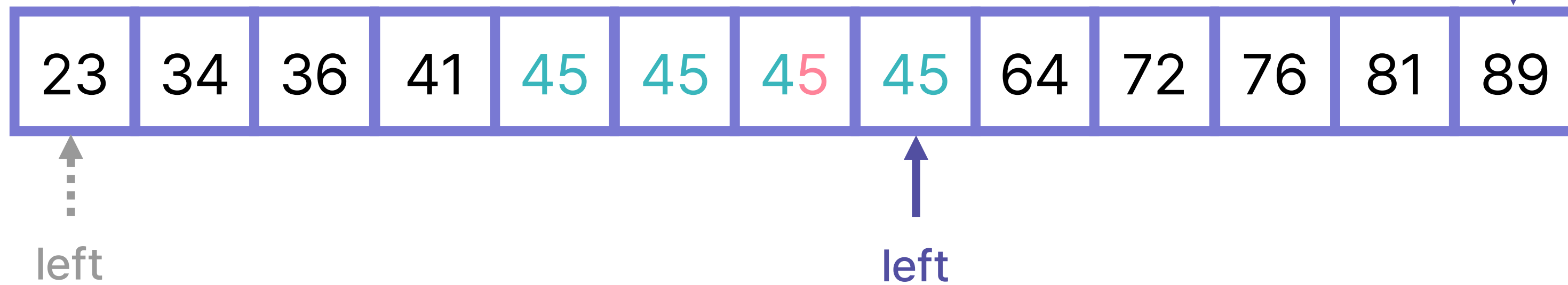
lower/upper bound

target: 45

left: $\text{mid} + 1 = 7$

right: 12

mid: 6



현재 **mid** 값이 우리가 찾는 값과 같으므로,
right가 아닌 **left**를 움직여줌

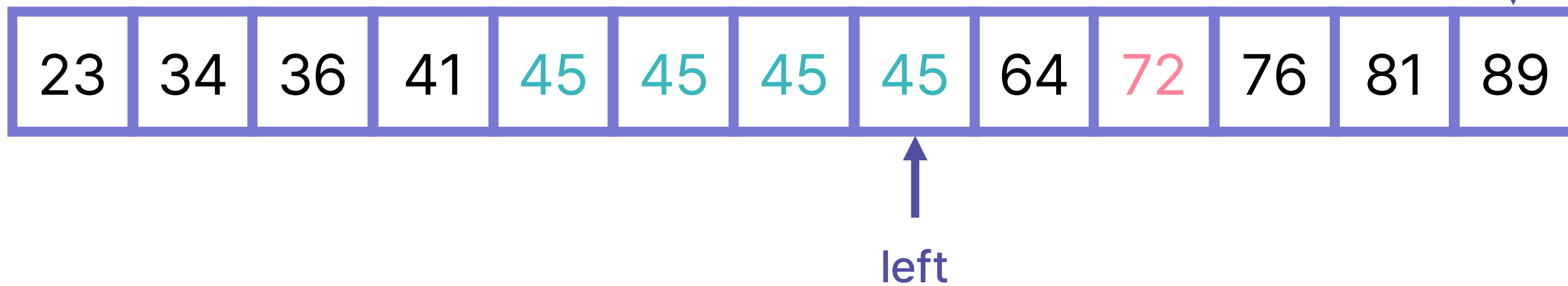
lower/upper bound

target: 45

left: 7

right: 12

mid: $(7 + 12) / 2 = 9$



같은 때 **left**를 움직이는 것 빼면,

나머지는 모두 기존 이진 탐색 알고리즘과 동일하게 동작함

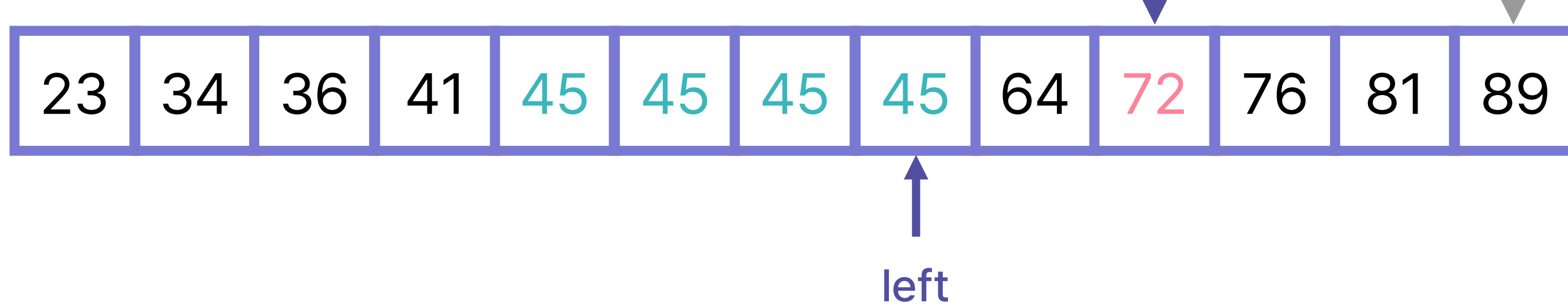
lower/upper bound

target: 45

left: 7

right: mid = 9

mid: 9



같은 때 **left**를 움직이는 것 빼면,

나머지는 모두 기존 이진 탐색 알고리즘과 동일하게 동작함

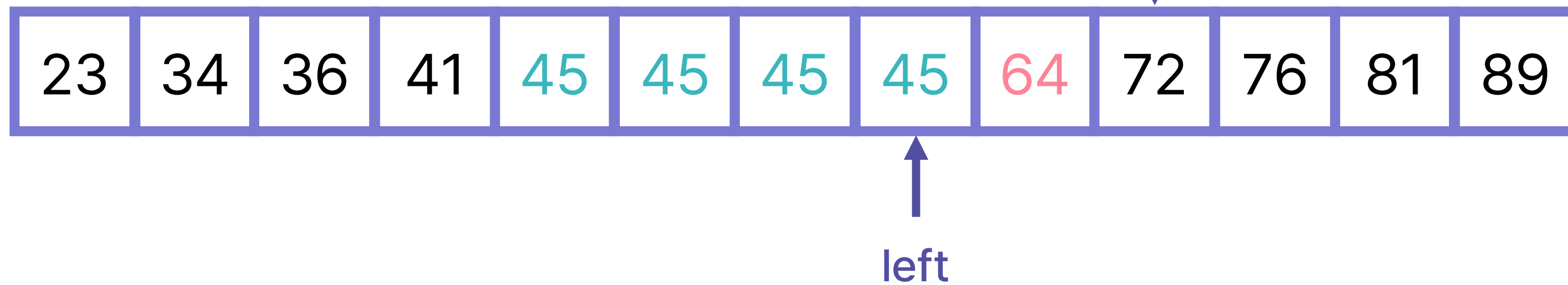
lower/upper bound

target: 45

left: 7

right: 9

mid: $(7 + 9) / 2 = 8$



같은 때 **left**를 움직이는 것 빼면,

나머지는 모두 기존 이진 탐색 알고리즘과 동일하게 동작함

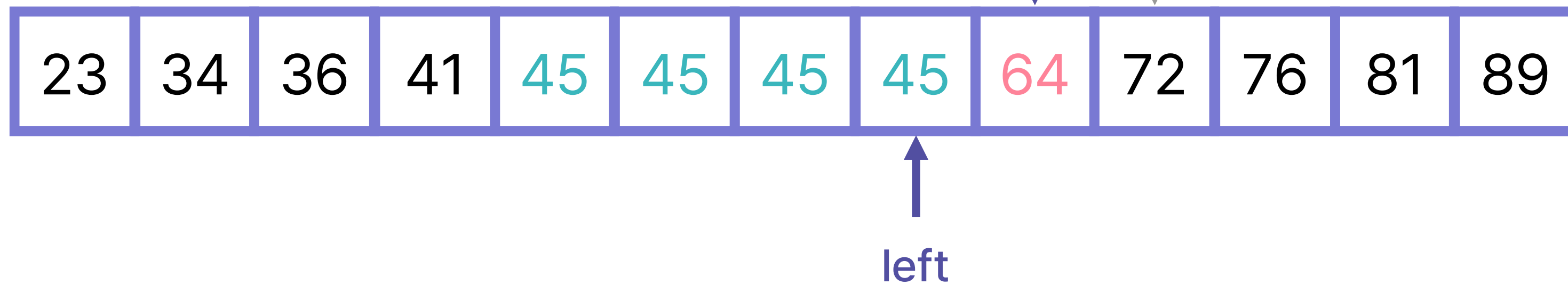
lower/upper bound

target: 45

left: 7

right: mid = 8

mid: 8



같은 때 **left**를 움직이는 것 빼면,

나머지는 모두 기존 이진 탐색 알고리즘과 동일하게 동작함

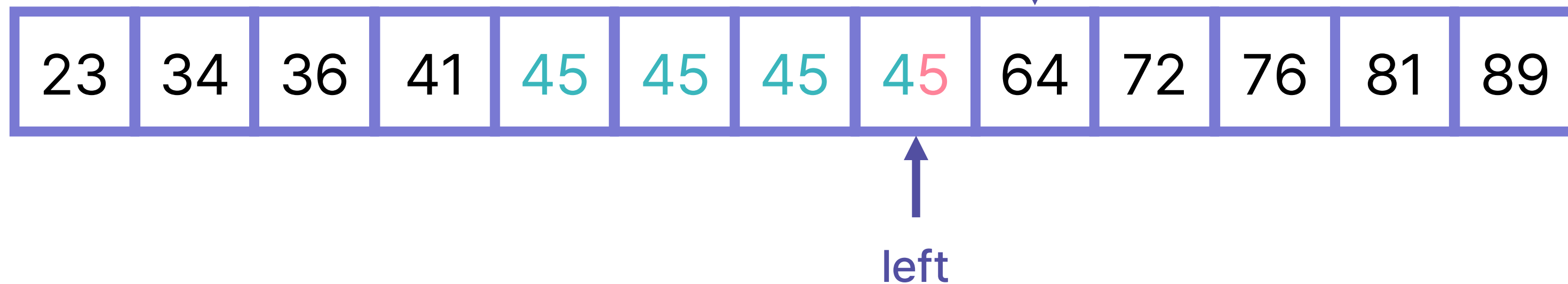
lower/upper bound

target: 45

left: 7

right: 8

mid: $(7 + 8) / 2 = 7$



같은 때 **left**를 움직이는 것 빼면,

나머지는 모두 기존 이진 탐색 알고리즘과 동일하게 동작함

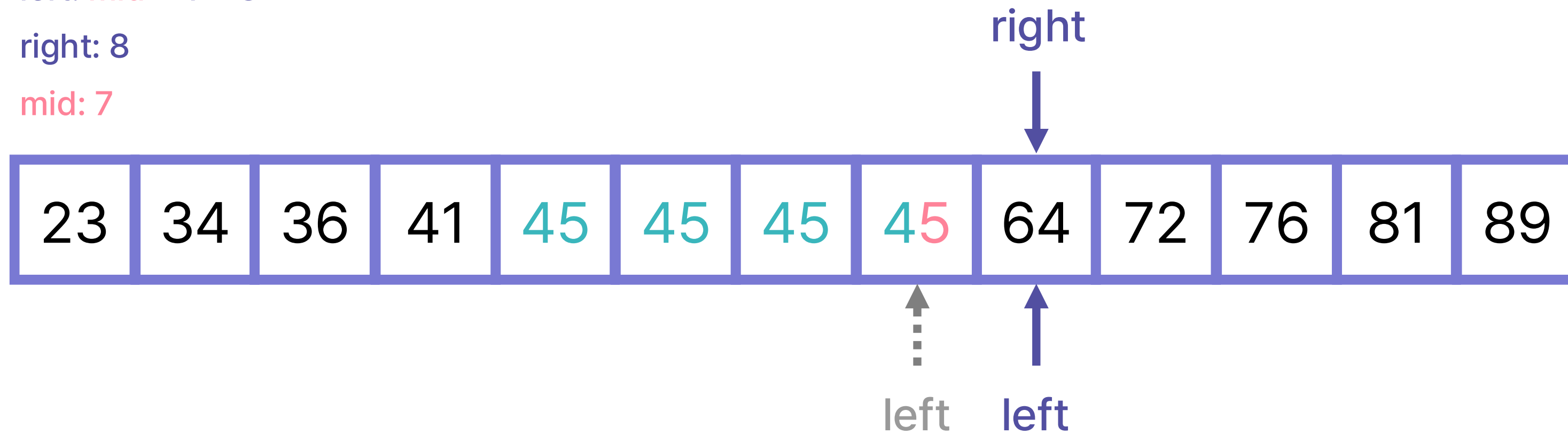
lower/upper bound

target: 45

left: $\text{mid} + 1 = 8$

right: 8

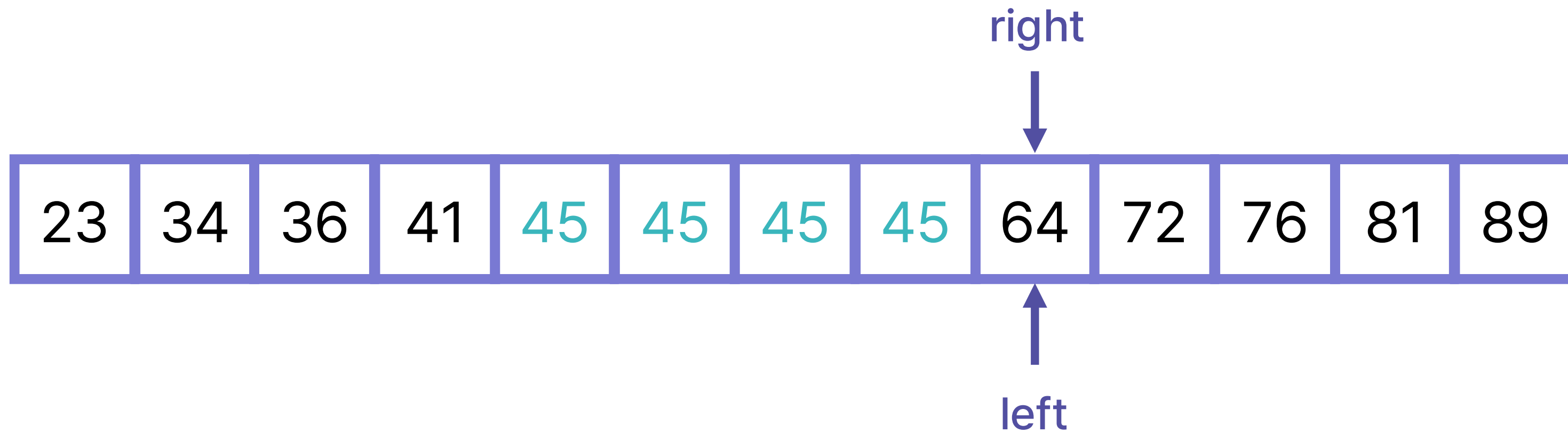
mid: 7



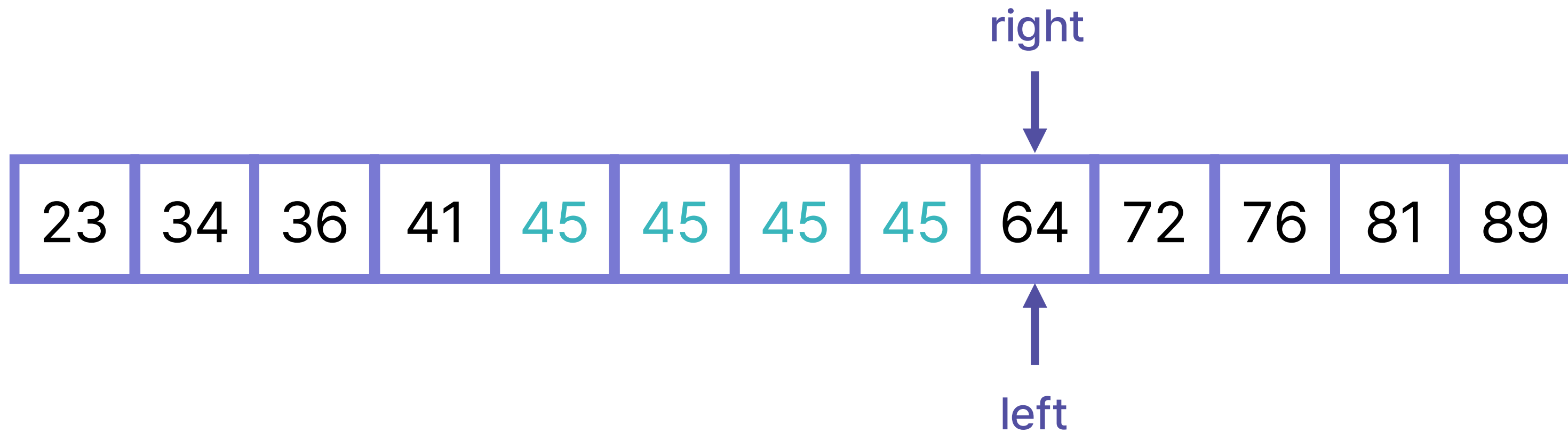
같은 때 **left**를 움직이는 것 빼면,

나머지는 모두 기존 이진 탐색 알고리즘과 동일하게 동작함

lower/upper bound



이렇게 하면 우리가 찾는 숫자 바깥을 가리키게 됨

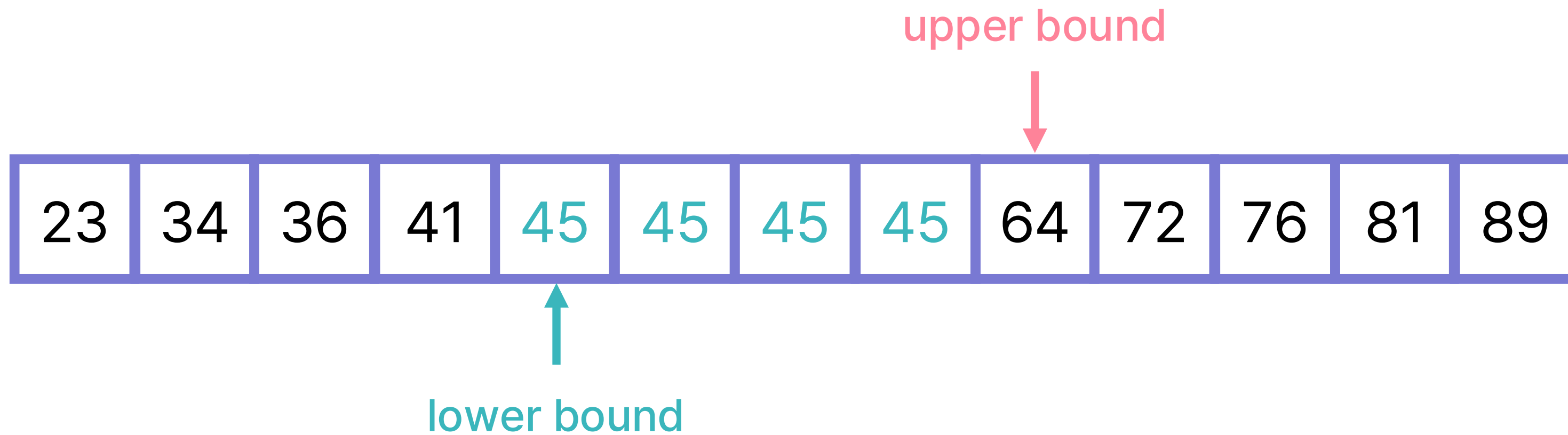


실제로, 이런 식으로 탐색을 진행하게 되면
우리가 찾는 값보다 큰 수 중 가장 작은 수를 가리킴



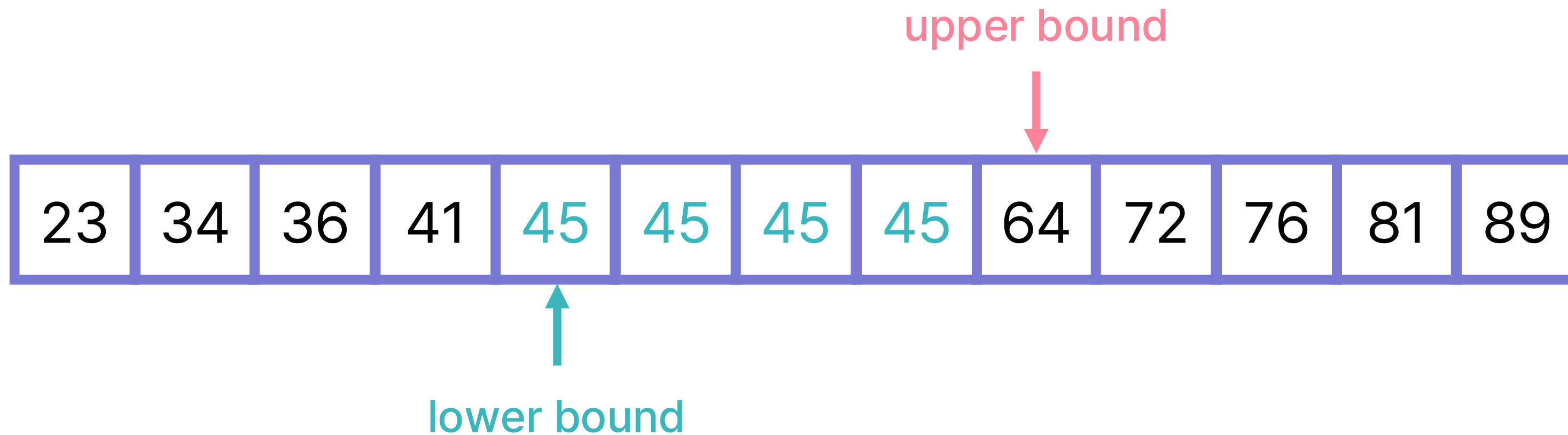
정리하면, 우리가 시도한 두 방법의 경우
각각 다음과 같은 위치를 가리키는 것을 볼 수 있음

lower/upper bound



우리는 방법 1에서 찾은 위치를 **lower bound** 라고 부르고,
방법 2로 찾은 위치를 **upper bound** 라고 부름

lower/upper bound



lower bound 는 찾는 값 이상인 첫 값의 인덱스를 가리키고,
upper bound 는 찾는 값을 초과하는 첫 값의 인덱스를 가리킴



lower/upper bound

• 정렬된 배열 v 에 대해

- $lower_bound(v, v + n, x)$ 는 $v[i - 1] < x \leq v[i]$ 인 i 를 반환하는 함수
 - $[v \sim v + n)$ 범위에서 값이 x 이상인, 처음 등장하는 $index$ 를 반환
- $upper_bound(v, v + n, x)$ 는 $v[i - 1] \leq x < v[i]$ 인 i 를 반환하는 함수
 - $[v \sim v + n)$ 범위에서 값이 x 초과인, 처음 등장하는 $index$ 를 반환

```
def lower_bound(arr, key):  
    # arr 내에서 key 보다 크거나 같은 첫 번째 원소의  
    # 인덱스를 찾음  
    index = bisect.bisect_left(arr, key)  
    return index  
  
def upper_bound(arr, key):  
    # arr 내에서 key 보다 큰 첫 번째 원소의 인덱스를 찾음  
    index = bisect.bisect_right(arr,  
    key)  
    return index
```

구현 방법은 고정이지 아니라 다양함!

*bisect*를 활용한 Python 코드